

Squad RPG Analysis

Early Player Engagement, Retention, and Behavioral Segmentation

DSO 567 · Data Project 1 · 2026

Private course analysis. The assignment restricts the dataset and data specification to course use. Review permissions before public release.

Editorial revision: section headings standardized; original analysis code, interpretations, and saved outputs retained. Results have not been rerun.

1. Setup and Data Loading

Code cell 3 · saved execution 1

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

Code cell 4 · saved execution 2

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import re
from itertools import combinations
```

Code cell 5 · saved execution 3

```
from matplotlib.ticker import PercentFormatter
sns.set_theme(style='whitegrid', context='notebook')
plt.rcParams.update({'figure.dpi': 110, 'axes.titleweight': 'bold',
                    'axes.spines.top': False, 'axes.spines.right': False})

SEG_COLORS = {'Retained Spender': '#1b9e77', 'Retained Non-spender': '#7570b3',
              'Lost Spender': '#d95f02', 'Lost Non-spender': '#9e9e9e'}

def cluster_colors(labels):
    """colour per cluster id; DBSCAN noise (-1) is always light grey."""
    pal = sns.color_palette('tab10')
    return {k: ('#c7c7c7' if k == -1 else pal[int(k) % 10]) for k in labels}
```

Code cell 6 · saved execution 4

```
# Display all columns
pd.set_option('display.max_columns', None)
```

Code cell 7 · saved execution 42

```
personal_path = '/content/drive/My Drive'

# dont change this file path
file_path = '/Data Project 1 - DSO 567/Data/dso567_data_project1.csv'

df = pd.read_csv("/content/drive/MyDrive/Data Project 1 - DSO 567/Data/dso567_data_project1.csv")
df.head()
```

	Player ID	INSTL_TS_UTC	AD_NETWORK	AD_CHANNEL	COUNTRY	\
0	Player1	2022-05-28T09:44:40.819Z	Unattributed	Paid	SE	
1	Player2	2022-05-30T15:39:33.172Z	Organic	Organic	FI	
2	Player3	2022-05-20T06:52:03.125Z	Unity	Paid	AU	
3	Player4	2022-03-25T23:35:31.306Z	Unattributed	Paid	AU	
4	Player5	2022-02-09T12:13:25.088Z	Unattributed	Paid	PH	

	CITY	LANGUAGE	DEVICE_BRAND	\
0	Kristianstad	Swedish	HUAWEI	
1	Helsinki	Finnish	OnePlus	
2	Perth	English	OPPO	
3	Sydney	English	samsung	
4	Quezon City	English	realme	

	DX_TOTAL_SESSION_CT	\
0	[\n 1, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	
1	[\n 1, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	
2	[\n 1, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	
3	[\n 1, \n 4, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	
4	[\n 1, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	

	DX_TOTAL_PLAYTIME	\
0	[\n 1.77, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	
1	[\n 19.77, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	
2	[\n 5.02, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n 0, \n ...	

```

3 [\n 1.43,\n 43.28,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 14.67,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_TOTAL_SPEND \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_CAMPAIGN_MISSIONS_WON \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 2,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 2,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_CAMPAIGN_MISSIONS_LOST \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_CAMPAIGN_MISSIONS_RETREAT \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_PVP_MATCHES_WON \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_PVP_MATCHES_LOST \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_PVP_MATCHES_RETREAT \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_SIDE_MISSIONS_WON \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_SIDE_MISSIONS_LOST \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_SIDE_MISSIONS_RETREAT \
0 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_START_ISIN_CHAPTER \
0 [\n undefined,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 1,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n undefined,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 1,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

                                DX_END_ISIN_CHAPTER
0 [\n undefined,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
1 [\n 1,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
2 [\n undefined,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
3 [\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...
4 [\n 1,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n 0,\n ...

```

2. Reusable Analysis Functions

Code cell 9 · saved execution 43

```

# offsets
days = ['0', '1', '3', '7', '14', '21', '30', '60', '90', '120', '180', '240', '300', '365']

def parse_list_col(series):
    """Turn a column of stringified lists like '[1, 0, 0]' into a column of numeric lists."""
    return (
        series
        .str.replace(r'[\n\\]', '', regex=True)
        .str.split(r',\s*')
        .apply(lambda lst: [pd.to_numeric(v, errors='coerce') for v in lst] if isinstance(lst, list) else None)
    )

def get_day_cols(df, metric, max_day=None):
    """
    Return columns like D0_<metric>, D1_<metric>, ... sorted by day number.
    If max_day is set, only keep days 0..max_day (e.g. max_day=1 -> D0 and D1).
    """
    pattern = re.compile(rf'^D(\d+)_([re.escape(metric)])$')
    matched = []
    for col in df.columns:
        m = pattern.match(col)
        if m:
            day = int(m.group(1))
            if max_day is None or day <= max_day:
                matched.append((day, col))
    matched.sort()
    return [col for _, col in matched]

def plot_distribution(df, col, bins=30, log=False):
    """Histogram of a single numeric column's distribution."""
    fig, ax = plt.subplots(figsize=(7, 4))
    data = df[col].dropna()
    ax.hist(data, bins=bins, edgecolor='black', alpha=0.7)
    if log:
        ax.set_yscale('log')
    ax.set_title(f'Distribution: {col}')
    ax.set_xlabel(col)
    ax.set_ylabel('Count')
    plt.tight_layout()
    plt.show()

def plot_metric_trend(df, metric, agg='mean'):
    """
    Line plot showing how a metric evolves across D0, D1, D3 ... D365.
    agg: 'mean', 'median', or 'sum'
    """
    cols = get_day_cols(df, metric)
    if not cols:
        print(f'No columns found for metric: {metric}')
        return

    if agg == 'mean':
        values = df[cols].mean()
    elif agg == 'median':
        values = df[cols].median()
    elif agg == 'sum':
        values = df[cols].sum()
    else:
        raise ValueError("agg must be 'mean', 'median', or 'sum'")

    x_labels = [c.split('_')[0].replace('D', '') for c in cols] # '0','1','3',...

    fig, ax = plt.subplots(figsize=(9, 4))
    ax.plot(x_labels, values.values, marker='o')
    ax.set_title(f'{agg.capitalize()} of {metric} over time')
    ax.set_xlabel('Day')
    ax.set_ylabel(f'{agg.capitalize()} {metric}')
    plt.tight_layout()
    plt.show()

def plot_missing_by_day(df, metric):
    """Bar chart of NaN counts per day column for a given metric – good for spotting data gaps."""
    cols = get_day_cols(df, metric)
    if not cols:
        print(f'No columns found for metric: {metric}')
        return

    missing = df[cols].isna().sum()
    x_labels = [c.split('_')[0].replace('D', '') for c in cols]

    fig, ax = plt.subplots(figsize=(9, 4))
    ax.bar(x_labels, missing.values, color='indianred')
    ax.set_title(f'Missing values by day: {metric}')
    ax.set_xlabel('Day')
    ax.set_ylabel('NaN count')
    plt.tight_layout()
    plt.show()

def plot_all_metrics_trend(df, metrics, agg='mean'):

```

```

"""Overlay multiple metrics' trends on one plot (useful when comparing WON/LOST/RETREAT together)."""
fig, ax = plt.subplots(figsize=(10, 5))
for metric in metrics:
    cols = get_day_cols(df, metric)
    if not cols:
        continue
    values = df[cols].mean() if agg == 'mean' else df[cols].sum()
    x_labels = [c.split('_')[0].replace('D', '') for c in cols]
    ax.plot(x_labels, values.values, marker='o', label=metric)
ax.set_title(f'{agg.capitalize()} trend comparison')
ax.set_xlabel('Day')
ax.legend()
plt.tight_layout()
plt.show()

def summary_stats(df, metric):
    """Quick describe() table across all day columns for a metric."""
    cols = get_day_cols(df, metric)
    return df[cols].describe().T

def compare_segments(df, metric, segment_col='segment', stat='median',
                    segment_order=None, max_day=None, alpha=0.05,
                    correct=True, table=False, plot=True, min_n=5):
    """
    Compare a metric across day periods between all segments in `segment_col`,
    with pairwise tests per day.

    stat = 'median' -> medians + Mann-Whitney U (rank-biserial effect size)
    stat = 'mean' -> means + Welch's t-test (Cohen's d effect size)
    min_n = minimum number of valid (non-NaN) players required in EACH group
            to run a test; otherwise p_value / effect size are left as NaN.

    Returns (center_df, pair_df).
    """
    if stat not in ('median', 'mean'):
        raise ValueError("stat must be 'median' or 'mean'")

    cols = get_day_cols(df, metric, max_day=max_day)
    if not cols:
        print(f'No columns found for metric: {metric}')
        return None

    labels = list(segment_order) if segment_order is not None \
        else sorted(df[segment_col].dropna().unique())
    sub = df[df[segment_col].isin(labels)]
    sizes = sub[segment_col].value_counts()
    days_used = [c.split('_')[0] for c in cols]
    pairs = list(combinations(labels, 2))
    effect_name = 'rank_biserial' if stat == 'median' else 'cohens_d'

    # centers for every segment x day in one vectorized call
    center_df = (sub.groupby(segment_col)[cols].agg(stat)
                .reindex(labels).T)
    center_df.index = days_used
    center_df.columns = [f'{lab}_{stat}' for lab in labels]
    center_df = center_df.rename_axis('day').reset_index()

    # split once: {segment: DataFrame of day columns}
    groups = {lab: sub.loc[sub[segment_col] == lab, cols] for lab in labels}

    pair_rows = []
    for c, d in zip(cols, days_used):
        vals = {lab: groups[lab][c].dropna() for lab in labels}
        for a, b in pairs:
            va, vb = vals[a], vals[b]

            # too few players in one group -> skip the test, keep the sample sizes
            if len(va) < min_n or len(vb) < min_n:
                pair_rows.append({'day': d, 'group_a': a, 'group_b': b,
                                'n_a': len(va), 'n_b': len(vb),
                                f'{stat}_a': va.agg(stat) if len(va) else np.nan,
                                f'{stat}_b': vb.agg(stat) if len(vb) else np.nan,
                                'p_value': np.nan, effect_name: np.nan})
                continue

            if stat == 'median':
                u, p = stats.mannwhitneyu(va, vb, alternative='two-sided', method='asymptotic')
                effect = 2 * u / (len(va) * len(vb)) - 1 # rank-biserial
                row = {'median_a': va.median(), 'median_b': vb.median(),
                      'p_value': p, 'rank_biserial': effect}
            else:
                _, p = stats.ttest_ind(va, vb, equal_var=False)
                pooled = np.sqrt((va.std()**2 + vb.std()**2) / 2)
                effect = (va.mean() - vb.mean()) / pooled if pooled > 0 else np.nan
                row = {'mean_a': va.mean(), 'mean_b': vb.mean(),
                      'p_value': p, 'cohens_d': effect}

            pair_rows.append({'day': d, 'group_a': a, 'group_b': b,
                              'n_a': len(va), 'n_b': len(vb), **row})

    pair_df = pd.DataFrame(pair_rows)
    pair_df['p_adj'] = (pair_df['p_value'] * len(pairs)).clip(upper=1) if correct else pair_df['p_value']

```

```

pair_df['significant'] = pair_df['p_adj'] < alpha # NaN p-values -> False
center_df['any_significant'] = center_df['day'].map(
    pair_df.groupby('day', sort=False)['significant'].any())

if plot:
    fig, ax = plt.subplots(figsize=(11, 5))
    for lab in labels:
        ax.plot(center_df['day'], center_df[f'{lab}_{stat}'], marker='o',
            label=f'{lab} (n={sizes[lab]:,})')
        ccols = [f'{lab}_{stat}' for lab in labels]
        y_max = center_df[ccols].max().max()
        y_max = y_max if y_max > 0 else 1
        for i, row in center_df.iterrows():
            if row['any_significant']:
                ax.annotate('!', (i, row[ccols].max() + y_max * 0.03),
                    ha='center', fontsize=16)
        adj = 'Bonferroni-adjusted ' if correct else ''
        test = 'Mann-Whitney U' if stat == 'median' else 'Welch's t-test'
        ax.set_title(f'{stat.capitalize()} of {metric} by segment\n'
            f'(* = at least one pair differs, {adj}p<{alpha}, {test}; '
            f'pairs with n<{min_n} not tested)')
        ax.set_xlabel('Day'); ax.set_ylabel(f'{stat.capitalize()} {metric}')
        ax.legend(); plt.tight_layout(); plt.show()

if table:
    display(center_df); display(pair_df)

return center_df, pair_df

# thin wrappers so existing calls keep working
def compare_segments_median_test(df, metric, **kw):
    return compare_segments(df, metric, stat='median', **kw)

def compare_segments_mean_ttest(df, metric, **kw):
    return compare_segments(df, metric, stat='mean', **kw)

```

thin wrappers so your existing calls keep working (only the first argument changes)

```

def compare_segments_median_test(df, metric, **kw):
    return compare_segments(df, metric, stat='median', **kw)

def compare_segments_mean_ttest(df, metric, **kw):
    return compare_segments(df, metric, stat='mean', **kw)

```

Code cell 11 · saved execution 44

```

# cal_cols = ['PVP_MATCHES', 'CAMPAIGN_MISSIONS', 'SIDE_MISSIONS']

# for cal in cal_cols:
#     df['DX_TOTAL_' + cal] = df['DX_' + cal + '_WON'] + df['DX_' + cal + '_LOST'] + df['DX_' + cal + '_RETREAT']

# df.head()

```

3. Data Preparation and Feature Engineering

3.1. Parse Daily Arrays and Calculate Gameplay Totals

Code cell 13 · saved execution 10

```

days = ['0', '1', '3', '7', '14', '21', '30', '60', '90', '120', '180', '240', '300', '365']

dx_cols = df.columns[df.columns.str.contains(r'^DX_', regex=True)]

# Step 1: parse every DX_ column into a column of numeric lists
parsed_cols = {col: parse_list_col(df[col]) for col in dx_cols}

# Step 2: build the DX_TOTAL_* list columns by summing WON + LOST + RETREAT element-wise
cal_cols = ['PVP_MATCHES', 'CAMPAIGN_MISSIONS', 'SIDE_MISSIONS']

for cal in cal_cols:
    won = parsed_cols['DX_' + cal + '_WON']
    lost = parsed_cols['DX_' + cal + '_LOST']
    retreat = parsed_cols['DX_' + cal + '_RETREAT']

    totals = []

    for won_row, lost_row, retreat_row in zip(won, lost, retreat):
        if isinstance(won_row, list) and isinstance(lost_row, list) and isinstance(retreat_row, list):
            row_total = []
            for w, l, r in zip(won_row, lost_row, retreat_row):
                row_total.append(w + l + r)
            totals.append(row_total)
        else:
            totals.append(None)

    parsed_cols['DX_TOTAL_' + cal] = totals

# Step 3: expand every parsed list column (original + new totals) into D{day}_ columns

```

```

new_cols = {}

for col, parsed in parsed_cols.items():
    for i, d in enumerate(days):
        name = col.replace('DX', 'D' + d)
        new_cols[name] = pd.Series(parsed, index=df.index).apply(
            lambda x: x[i] if isinstance(x, list) and len(x) > i else None
        )

new_df = pd.DataFrame(new_cols, index=df.index)

# already numeric from parse_list_col, but coerce again to be safe (handles any leftover None/object dtype)
new_df = new_df.apply(pd.to_numeric, errors='coerce')

cleaned_df = pd.concat([df, new_df], axis=1)

# drop original DX cols
cleaned_df.drop(columns=dx_cols, inplace=True)

cleaned_df.head()

```

	Player ID	INSTL_TS_UTC	AD_NETWORK	AD_CHANNEL	COUNTRY	\
0	Player1	2022-05-28T09:44:40.819Z	Unattributed	Paid	SE	
1	Player2	2022-05-30T15:39:33.172Z	Organic	Organic	FI	
2	Player3	2022-05-20T06:52:03.125Z	Unity	Paid	AU	
3	Player4	2022-03-25T23:35:31.306Z	Unattributed	Paid	AU	
4	Player5	2022-02-09T12:13:25.088Z	Unattributed	Paid	PH	

	CITY	LANGUAGE	DEVICE_BRAND	D0_TOTAL_SESSION_CT	\
0	Kristianstad	Swedish	HUAWEI	1	
1	Helsinki	Finnish	OnePlus	1	
2	Perth	English	OPPO	1	
3	Sydney	English	samsung	1	
4	Quezon City	English	realme	1	

	D1_TOTAL_SESSION_CT	D3_TOTAL_SESSION_CT	D7_TOTAL_SESSION_CT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	4	0	0	
4	0	0	0	

	D14_TOTAL_SESSION_CT	D21_TOTAL_SESSION_CT	D30_TOTAL_SESSION_CT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D60_TOTAL_SESSION_CT	D90_TOTAL_SESSION_CT	D120_TOTAL_SESSION_CT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D180_TOTAL_SESSION_CT	D240_TOTAL_SESSION_CT	D300_TOTAL_SESSION_CT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D365_TOTAL_SESSION_CT	D0_TOTAL_PLAYTIME	D1_TOTAL_PLAYTIME	\
0	0	1.77	0.00	
1	0	19.77	0.00	
2	0	5.02	0.00	
3	0	1.43	43.28	
4	0	14.67	0.00	

	D3_TOTAL_PLAYTIME	D7_TOTAL_PLAYTIME	D14_TOTAL_PLAYTIME	\
0	0.0	0.0	0.0	
1	0.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	

	D21_TOTAL_PLAYTIME	D30_TOTAL_PLAYTIME	D60_TOTAL_PLAYTIME	\
0	0.0	0.0	0.0	
1	0.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	

	D90_TOTAL_PLAYTIME	D120_TOTAL_PLAYTIME	D180_TOTAL_PLAYTIME	\
0	0.0	0.0	0.0	
1	0.0	0.0	0.0	
2	0.0	0.0	0.0	
3	0.0	0.0	0.0	
4	0.0	0.0	0.0	

	D240_TOTAL_PLAYTIME	D300_TOTAL_PLAYTIME	D365_TOTAL_PLAYTIME	\
--	---------------------	---------------------	---------------------	---

0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0
3	0.0	0.0	0.0
4	0.0	0.0	0.0

	D0_TOTAL_SPEND	D1_TOTAL_SPEND	D3_TOTAL_SPEND	D7_TOTAL_SPEND	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

	D14_TOTAL_SPEND	D21_TOTAL_SPEND	D30_TOTAL_SPEND	D60_TOTAL_SPEND	\
0	0.0	0.0	0.0	0.0	
1	0.0	0.0	0.0	0.0	
2	0.0	0.0	0.0	0.0	
3	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	0.0	

	D90_TOTAL_SPEND	D120_TOTAL_SPEND	D180_TOTAL_SPEND	D240_TOTAL_SPEND	\
0	0.0	0.0	0	0.0	
1	0.0	0.0	0	0.0	
2	0.0	0.0	0	0.0	
3	0.0	0.0	0	0.0	
4	0.0	0.0	0	0.0	

	D300_TOTAL_SPEND	D365_TOTAL_SPEND	D0_CAMPAIGN_MISSIONS_WON	\
0	0	0.0	0	
1	0	0.0	2	
2	0	0.0	0	
3	0	0.0	0	
4	0	0.0	2	

	D1_CAMPAIGN_MISSIONS_WON	D3_CAMPAIGN_MISSIONS_WON	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D7_CAMPAIGN_MISSIONS_WON	D14_CAMPAIGN_MISSIONS_WON	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D21_CAMPAIGN_MISSIONS_WON	D30_CAMPAIGN_MISSIONS_WON	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D60_CAMPAIGN_MISSIONS_WON	D90_CAMPAIGN_MISSIONS_WON	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D120_CAMPAIGN_MISSIONS_WON	D180_CAMPAIGN_MISSIONS_WON	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D240_CAMPAIGN_MISSIONS_WON	D300_CAMPAIGN_MISSIONS_WON	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D365_CAMPAIGN_MISSIONS_WON	D0_CAMPAIGN_MISSIONS_LOST	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D1_CAMPAIGN_MISSIONS_LOST	D3_CAMPAIGN_MISSIONS_LOST	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D7_CAMPAIGN_MISSIONS_LOST	D14_CAMPAIGN_MISSIONS_LOST	\
0	0	0	

1	0	0
2	0	0
3	0	0
4	0	0

	D21_CAMPAIGN_MISSIONS_LOST	D30_CAMPAIGN_MISSIONS_LOST	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D60_CAMPAIGN_MISSIONS_LOST	D90_CAMPAIGN_MISSIONS_LOST	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D120_CAMPAIGN_MISSIONS_LOST	D180_CAMPAIGN_MISSIONS_LOST	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D240_CAMPAIGN_MISSIONS_LOST	D300_CAMPAIGN_MISSIONS_LOST	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D365_CAMPAIGN_MISSIONS_LOST	D0_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D1_CAMPAIGN_MISSIONS_RETREAT	D3_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D7_CAMPAIGN_MISSIONS_RETREAT	D14_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D21_CAMPAIGN_MISSIONS_RETREAT	D30_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D60_CAMPAIGN_MISSIONS_RETREAT	D90_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D120_CAMPAIGN_MISSIONS_RETREAT	D180_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D240_CAMPAIGN_MISSIONS_RETREAT	D300_CAMPAIGN_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D365_CAMPAIGN_MISSIONS_RETREAT	D0_PVP_MATCHES_WON	D1_PVP_MATCHES_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D3_PVP_MATCHES_WON	D7_PVP_MATCHES_WON	D14_PVP_MATCHES_WON	\
0	0	0	0	

1	0	0	0
2	0	0	0
3	0	0	0
4	0	0	0

	D21_PVP_MATCHES_WON	D30_PVP_MATCHES_WON	D60_PVP_MATCHES_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D90_PVP_MATCHES_WON	D120_PVP_MATCHES_WON	D180_PVP_MATCHES_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D240_PVP_MATCHES_WON	D300_PVP_MATCHES_WON	D365_PVP_MATCHES_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D0_PVP_MATCHES_LOST	D1_PVP_MATCHES_LOST	D3_PVP_MATCHES_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D7_PVP_MATCHES_LOST	D14_PVP_MATCHES_LOST	D21_PVP_MATCHES_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D30_PVP_MATCHES_LOST	D60_PVP_MATCHES_LOST	D90_PVP_MATCHES_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D120_PVP_MATCHES_LOST	D180_PVP_MATCHES_LOST	D240_PVP_MATCHES_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D300_PVP_MATCHES_LOST	D365_PVP_MATCHES_LOST	D0_PVP_MATCHES_RETREAT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D1_PVP_MATCHES_RETREAT	D3_PVP_MATCHES_RETREAT	D7_PVP_MATCHES_RETREAT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D14_PVP_MATCHES_RETREAT	D21_PVP_MATCHES_RETREAT	D30_PVP_MATCHES_RETREAT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D60_PVP_MATCHES_RETREAT	D90_PVP_MATCHES_RETREAT	D120_PVP_MATCHES_RETREAT	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D180_PVP_MATCHES_RETREAT	D240_PVP_MATCHES_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D300_PVP_MATCHES_RETREAT	D365_PVP_MATCHES_RETREAT	D0_SIDE_MISSIONS_WON	\
0	0	0	0	
1	0	0	0	

2	0	0	0
3	0	0	0
4	0	0	0

	D1_SIDE_MISSTIONS_WON	D3_SIDE_MISSTIONS_WON	D7_SIDE_MISSTIONS_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D14_SIDE_MISSTIONS_WON	D21_SIDE_MISSTIONS_WON	D30_SIDE_MISSTIONS_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D60_SIDE_MISSTIONS_WON	D90_SIDE_MISSTIONS_WON	D120_SIDE_MISSTIONS_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D180_SIDE_MISSTIONS_WON	D240_SIDE_MISSTIONS_WON	D300_SIDE_MISSTIONS_WON	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D365_SIDE_MISSTIONS_WON	D0_SIDE_MISSTIONS_LOST	D1_SIDE_MISSTIONS_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D3_SIDE_MISSTIONS_LOST	D7_SIDE_MISSTIONS_LOST	D14_SIDE_MISSTIONS_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D21_SIDE_MISSTIONS_LOST	D30_SIDE_MISSTIONS_LOST	D60_SIDE_MISSTIONS_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D90_SIDE_MISSTIONS_LOST	D120_SIDE_MISSTIONS_LOST	D180_SIDE_MISSTIONS_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D240_SIDE_MISSTIONS_LOST	D300_SIDE_MISSTIONS_LOST	D365_SIDE_MISSTIONS_LOST	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D0_SIDE_MISSTIONS_RETREAT	D1_SIDE_MISSTIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D3_SIDE_MISSTIONS_RETREAT	D7_SIDE_MISSTIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D14_SIDE_MISSTIONS_RETREAT	D21_SIDE_MISSTIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D30_SIDE_MISSTIONS_RETREAT	D60_SIDE_MISSTIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	

3	0	0
4	0	0

	D90_SIDE_MISSIONS_RETREAT	D120_SIDE_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D180_SIDE_MISSIONS_RETREAT	D240_SIDE_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D300_SIDE_MISSIONS_RETREAT	D365_SIDE_MISSIONS_RETREAT	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D0_START_ISIN_CHAPTER	D1_START_ISIN_CHAPTER	D3_START_ISIN_CHAPTER	\
0	NaN	0.0	0	
1	1.0	0.0	0	
2	NaN	0.0	0	
3	0.0	0.0	0	
4	1.0	0.0	0	

	D7_START_ISIN_CHAPTER	D14_START_ISIN_CHAPTER	D21_START_ISIN_CHAPTER	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D30_START_ISIN_CHAPTER	D60_START_ISIN_CHAPTER	D90_START_ISIN_CHAPTER	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D120_START_ISIN_CHAPTER	D180_START_ISIN_CHAPTER	D240_START_ISIN_CHAPTER	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D300_START_ISIN_CHAPTER	D365_START_ISIN_CHAPTER	D0_END_ISIN_CHAPTER	\
0	0	0	NaN	
1	0	0	1.0	
2	0	0	NaN	
3	0	0	0.0	
4	0	0	1.0	

	D1_END_ISIN_CHAPTER	D3_END_ISIN_CHAPTER	D7_END_ISIN_CHAPTER	\
0	0.0	0	0	
1	0.0	0	0	
2	0.0	0	0	
3	0.0	0	0	
4	0.0	0	0	

	D14_END_ISIN_CHAPTER	D21_END_ISIN_CHAPTER	D30_END_ISIN_CHAPTER	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D60_END_ISIN_CHAPTER	D90_END_ISIN_CHAPTER	D120_END_ISIN_CHAPTER	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D180_END_ISIN_CHAPTER	D240_END_ISIN_CHAPTER	D300_END_ISIN_CHAPTER	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D365_END_ISIN_CHAPTER	D0_TOTAL_PVP_MATCHES	D1_TOTAL_PVP_MATCHES	\
0	0	0	0	
1	0	0	0	
2	0	0	0	

3	0	0	0
4	0	0	0

	D3_TOTAL_PVP_MATCHES	D7_TOTAL_PVP_MATCHES	D14_TOTAL_PVP_MATCHES	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D21_TOTAL_PVP_MATCHES	D30_TOTAL_PVP_MATCHES	D60_TOTAL_PVP_MATCHES	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D90_TOTAL_PVP_MATCHES	D120_TOTAL_PVP_MATCHES	D180_TOTAL_PVP_MATCHES	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D240_TOTAL_PVP_MATCHES	D300_TOTAL_PVP_MATCHES	D365_TOTAL_PVP_MATCHES	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D0_TOTAL_CAMPAIGN_MISSIONS	D1_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	2	0	
2	0	0	
3	0	0	
4	2	0	

	D3_TOTAL_CAMPAIGN_MISSIONS	D7_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D14_TOTAL_CAMPAIGN_MISSIONS	D21_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D30_TOTAL_CAMPAIGN_MISSIONS	D60_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D90_TOTAL_CAMPAIGN_MISSIONS	D120_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D180_TOTAL_CAMPAIGN_MISSIONS	D240_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D300_TOTAL_CAMPAIGN_MISSIONS	D365_TOTAL_CAMPAIGN_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D0_TOTAL_SIDE_MISSIONS	D1_TOTAL_SIDE_MISSIONS	D3_TOTAL_SIDE_MISSIONS	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D7_TOTAL_SIDE_MISSIONS	D14_TOTAL_SIDE_MISSIONS	D21_TOTAL_SIDE_MISSIONS	\
0	0	0	0	
1	0	0	0	
2	0	0	0	

3	0	0	0
4	0	0	0

	D30_TOTAL_SIDE_MISSIONS	D60_TOTAL_SIDE_MISSIONS	D90_TOTAL_SIDE_MISSIONS	\
0	0	0	0	
1	0	0	0	
2	0	0	0	
3	0	0	0	
4	0	0	0	

	D120_TOTAL_SIDE_MISSIONS	D180_TOTAL_SIDE_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D240_TOTAL_SIDE_MISSIONS	D300_TOTAL_SIDE_MISSIONS	\
0	0	0	
1	0	0	
2	0	0	
3	0	0	
4	0	0	

	D365_TOTAL_SIDE_MISSIONS
0	0
1	0
2	0
3	0
4	0

3.2. Calculate Campaign Win Rates

Code cell 15 · saved execution 11

```
win_rate_cols = {}

for d in days:
    won = cleaned_df[f'D{d}_CAMPAIGN_MISSIONS_WON']
    total = cleaned_df[f'D{d}_TOTAL_CAMPAIGN_MISSIONS'] # WON + LOST + RETREAT

    # divide only where at least 1 mission was played; otherwise NaN (win rate undefined, not 0%)
    win_rate_cols[f'D{d}_CAMPAIGN_WIN_RATE'] = won / total.where(total > 0)

cleaned_df = pd.concat([cleaned_df, pd.DataFrame(win_rate_cols, index=cleaned_df.index)], axis=1)
```

4. Exploratory Data Analysis

Code cell 17 · saved execution 12

```
import seaborn as sns
import matplotlib.pyplot as plt

cat_cols = ['AD_NETWORK', 'AD_CHANNEL', 'COUNTRY', 'DEVICE_BRAND', 'CITY', 'LANGUAGE']

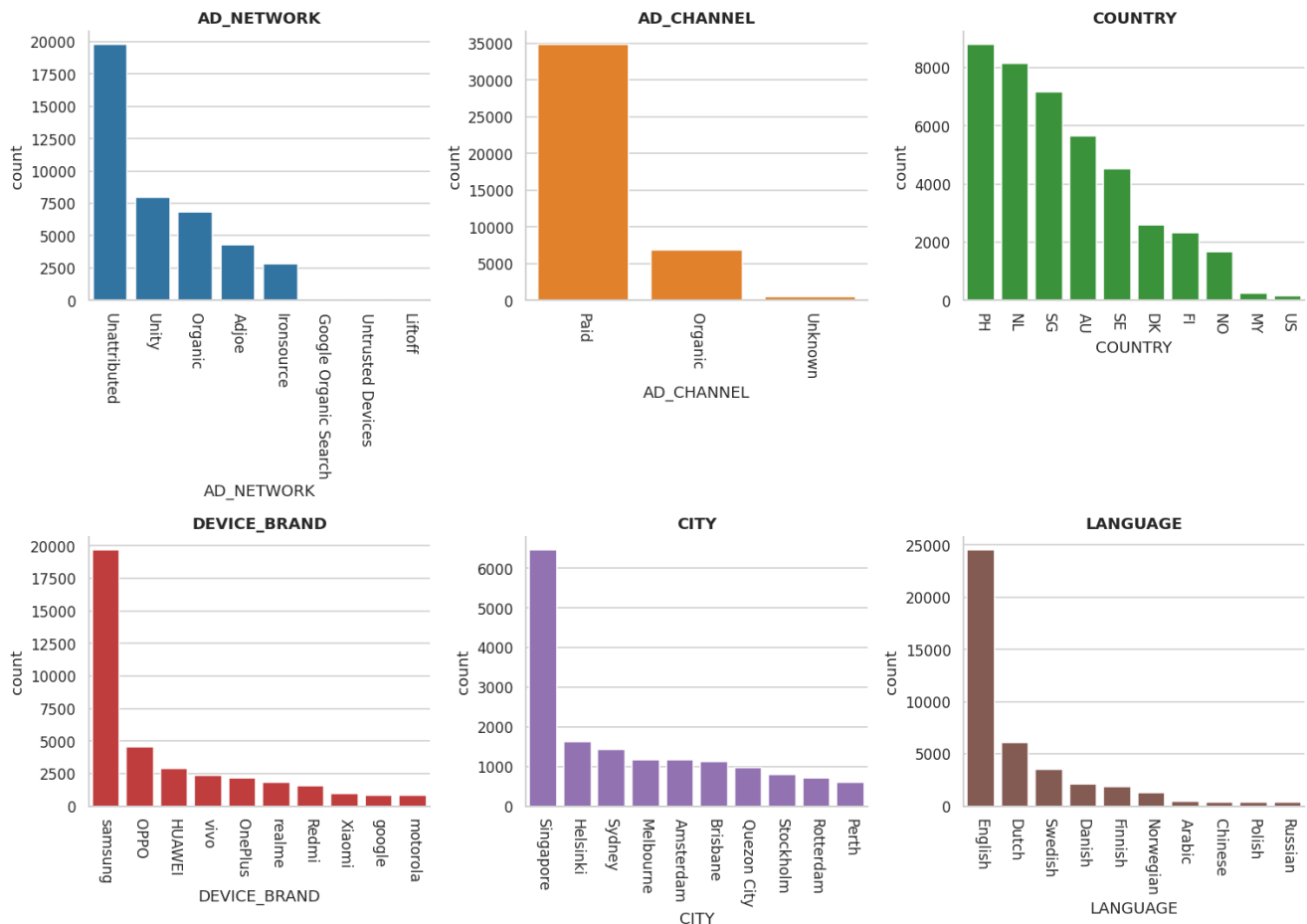
# Define a single solid color for each subplot to keep them distinct but uniform
column_colors = {
    'AD_NETWORK': '#1f77b4', # Steel Blue
    'AD_CHANNEL': '#ff7f0e', # Orange
    'COUNTRY': '#2ca02c', # Green
    'DEVICE_BRAND': '#d62728', # Red
    'CITY': '#9467bd', # Purple
    'LANGUAGE': '#8c564b' # Brown
}

fig, axes = plt.subplots(nrows=2, ncols=3, figsize=(14, 10))
axes = axes.flatten()

for i, col in enumerate(cat_cols):
    order = df[col].value_counts().head(10).index # sort bars by frequency
    color = column_colors.get(col, '#1f77b4')

    sns.countplot(
        data=df,
        x=col,
        order=order,
        ax=axes[i],
        color=color
    )
    axes[i].set_title(col)
    axes[i].tick_params(axis='x', rotation=-90)

plt.tight_layout()
plt.show()
```



4.1. Acquisition and Player Characteristics

Acquisition Network. The largest acquisition network is **Unattributed (19,823 players)**, followed by **Unity (7,954)** and **Organic (6,843)**. A large portion of the Android player base came through paid acquisition where the network could not be identified, or through major advertising networks. Because Unattributed installs are considered as Paid in the dataset and be interpreted as part of the paid acquisition population.

Acquisition Channel. Most players were acquired through **Paid channels (34,895; about 82.5%)**, compared with **Organic (6,899; about 16.3%)** and **Unknown (502; about 1.2%)**. The dataset is primarily players acquired through paid marketing. Organic players have higher average D0 sessions and playtime than paid players, while the Unknown group has the lowest averages.

Country. The largest acquisition locations come from the **Philippines (8,813)**, **Netherlands (8,137)**, **Singapore (7,162)**, **Australia (5,639)**, and **Sweden (4,528)**. The player sample is concentrated in several countries rather than evenly distributed worldwide.

City. **Singapore (6,471)** is the largest city, followed by **Helsinki (1,622)**, **Sydney (1,438)**, **Melbourne (1,167)**, and **Amsterdam (1,162)**. Singapore's large city count is consistent with Singapore also being one of the top countries in the dataset. Overall, city and countries are acquisition locations and should not automatically be interpreted as the locations with the highest overall demand for the game.

Language. **English (24,564 players, about 58%)** is the dominant device language, followed by Dutch, Swedish, Danish, Finnish, and Norwegian. Since Language represents the device's default language, it should not be interpreted directly as the player's nationality or native language.

Device Brand. **Samsung (19,705 players, about 46.6%)** is the largest device brand, followed by OPPO, HUAWEI, vivo, and OnePlus. Samsung users make up almost half of this data sample. As the population only includes Android users, it does not mean that the main people who play the game own Samsung devices.

Code cell 19 · saved execution 13

```
cleaned_df[cleaned_df['D0_TOTAL_SESSION_CT']==0]
```

	Player ID	INSTL_TS_UTC	AD_NETWORK	AD_CHANNEL	COUNTRY	\
31964	Player31965	2022-07-26T23:56:26.413Z	Unity	Paid	NL	
	CITY	LANGUAGE	DEVICE_BRAND	D0_TOTAL_SESSION_CT	\	
31964	Amsterdam	Spanish	samsung	0		
	D1_TOTAL_SESSION_CT	D3_TOTAL_SESSION_CT	D7_TOTAL_SESSION_CT	\		
31964	1	0	0			
	D14_TOTAL_SESSION_CT	D21_TOTAL_SESSION_CT	D30_TOTAL_SESSION_CT	\		
31964	0	0	0			
	D60_TOTAL_SESSION_CT	D90_TOTAL_SESSION_CT	D120_TOTAL_SESSION_CT	\		
31964	0	0	0			
	D180_TOTAL_SESSION_CT	D240_TOTAL_SESSION_CT	D300_TOTAL_SESSION_CT	\		
31964	0	0	0			
	D365_TOTAL_SESSION_CT	D0_TOTAL_PLAYTIME	D1_TOTAL_PLAYTIME	\		
31964	0	0.0	8.28			
	D3_TOTAL_PLAYTIME	D7_TOTAL_PLAYTIME	D14_TOTAL_PLAYTIME	\		
31964	0.0	0.0	0.0			
	D21_TOTAL_PLAYTIME	D30_TOTAL_PLAYTIME	D60_TOTAL_PLAYTIME	\		
31964	0.0	0.0	0.0			
	D90_TOTAL_PLAYTIME	D120_TOTAL_PLAYTIME	D180_TOTAL_PLAYTIME	\		
31964	0.0	0.0	0.0			
	D240_TOTAL_PLAYTIME	D300_TOTAL_PLAYTIME	D365_TOTAL_PLAYTIME	\		
31964	0.0	0.0	0.0			
	D0_TOTAL_SPEND	D1_TOTAL_SPEND	D3_TOTAL_SPEND	D7_TOTAL_SPEND	\	
31964	0.0	0.0	0.0	0.0		
	D14_TOTAL_SPEND	D21_TOTAL_SPEND	D30_TOTAL_SPEND	D60_TOTAL_SPEND	\	
31964	0.0	0.0	0.0	0.0		
	D90_TOTAL_SPEND	D120_TOTAL_SPEND	D180_TOTAL_SPEND	D240_TOTAL_SPEND	\	
31964	0.0	0.0	0	0.0		
	D300_TOTAL_SPEND	D365_TOTAL_SPEND	D0_CAMPAIGN_MISSIONS_WON	\		
31964	0	0.0	0			
	D1_CAMPAIGN_MISSIONS_WON	D3_CAMPAIGN_MISSIONS_WON	\			
31964	0	0				
	D7_CAMPAIGN_MISSIONS_WON	D14_CAMPAIGN_MISSIONS_WON	\			
31964	0	0				
	D21_CAMPAIGN_MISSIONS_WON	D30_CAMPAIGN_MISSIONS_WON	\			
31964	0	0				
	D60_CAMPAIGN_MISSIONS_WON	D90_CAMPAIGN_MISSIONS_WON	\			
31964	0	0				
	D120_CAMPAIGN_MISSIONS_WON	D180_CAMPAIGN_MISSIONS_WON	\			
31964	0	0				
	D240_CAMPAIGN_MISSIONS_WON	D300_CAMPAIGN_MISSIONS_WON	\			
31964	0	0				
	D365_CAMPAIGN_MISSIONS_WON	D0_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D1_CAMPAIGN_MISSIONS_LOST	D3_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D7_CAMPAIGN_MISSIONS_LOST	D14_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D21_CAMPAIGN_MISSIONS_LOST	D30_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D60_CAMPAIGN_MISSIONS_LOST	D90_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D120_CAMPAIGN_MISSIONS_LOST	D180_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D240_CAMPAIGN_MISSIONS_LOST	D300_CAMPAIGN_MISSIONS_LOST	\			
31964	0	0				
	D365_CAMPAIGN_MISSIONS_LOST	D0_CAMPAIGN_MISSIONS_RETREAT	\			
31964	0	0				
	D1_CAMPAIGN_MISSIONS_RETREAT	D3_CAMPAIGN_MISSIONS_RETREAT	\			
31964	0	0				
	D7_CAMPAIGN_MISSIONS_RETREAT	D14_CAMPAIGN_MISSIONS_RETREAT	\			
31964	0	0				

31964	D21_CAMPAIGN_MISSIONS_RETREAT	D30_CAMPAIGN_MISSIONS_RETREAT	\
	0	0	
31964	D60_CAMPAIGN_MISSIONS_RETREAT	D90_CAMPAIGN_MISSIONS_RETREAT	\
	0	0	
31964	D120_CAMPAIGN_MISSIONS_RETREAT	D180_CAMPAIGN_MISSIONS_RETREAT	\
	0	0	
31964	D240_CAMPAIGN_MISSIONS_RETREAT	D300_CAMPAIGN_MISSIONS_RETREAT	\
	0	0	
31964	D365_CAMPAIGN_MISSIONS_RETREAT	D0_PVP_MATCHES_WON	D1_PVP_MATCHES_WON \
	0	0	0
31964	D3_PVP_MATCHES_WON	D7_PVP_MATCHES_WON	D14_PVP_MATCHES_WON \
	0	0	0
31964	D21_PVP_MATCHES_WON	D30_PVP_MATCHES_WON	D60_PVP_MATCHES_WON \
	0	0	0
31964	D90_PVP_MATCHES_WON	D120_PVP_MATCHES_WON	D180_PVP_MATCHES_WON \
	0	0	0
31964	D240_PVP_MATCHES_WON	D300_PVP_MATCHES_WON	D365_PVP_MATCHES_WON \
	0	0	0
31964	D0_PVP_MATCHES_LOST	D1_PVP_MATCHES_LOST	D3_PVP_MATCHES_LOST \
	0	0	0
31964	D7_PVP_MATCHES_LOST	D14_PVP_MATCHES_LOST	D21_PVP_MATCHES_LOST \
	0	0	0
31964	D30_PVP_MATCHES_LOST	D60_PVP_MATCHES_LOST	D90_PVP_MATCHES_LOST \
	0	0	0
31964	D120_PVP_MATCHES_LOST	D180_PVP_MATCHES_LOST	D240_PVP_MATCHES_LOST \
	0	0	0
31964	D300_PVP_MATCHES_LOST	D365_PVP_MATCHES_LOST	D0_PVP_MATCHES_RETREAT \
	0	0	0
31964	D1_PVP_MATCHES_RETREAT	D3_PVP_MATCHES_RETREAT	D7_PVP_MATCHES_RETREAT \
	0	0	0
31964	D14_PVP_MATCHES_RETREAT	D21_PVP_MATCHES_RETREAT	\
	0	0	
31964	D30_PVP_MATCHES_RETREAT	D60_PVP_MATCHES_RETREAT	\
	0	0	
31964	D90_PVP_MATCHES_RETREAT	D120_PVP_MATCHES_RETREAT	\
	0	0	
31964	D180_PVP_MATCHES_RETREAT	D240_PVP_MATCHES_RETREAT	\
	0	0	
31964	D300_PVP_MATCHES_RETREAT	D365_PVP_MATCHES_RETREAT	\
	0	0	
31964	D0_SIDE_MISSIONS_WON	D1_SIDE_MISSIONS_WON	D3_SIDE_MISSIONS_WON \
	0	0	0
31964	D7_SIDE_MISSIONS_WON	D14_SIDE_MISSIONS_WON	D21_SIDE_MISSIONS_WON \
	0	0	0
31964	D30_SIDE_MISSIONS_WON	D60_SIDE_MISSIONS_WON	D90_SIDE_MISSIONS_WON \
	0	0	0
31964	D120_SIDE_MISSIONS_WON	D180_SIDE_MISSIONS_WON	D240_SIDE_MISSIONS_WON \
	0	0	0
31964	D300_SIDE_MISSIONS_WON	D365_SIDE_MISSIONS_WON	D0_SIDE_MISSIONS_LOST \
	0	0	0
31964	D1_SIDE_MISSIONS_LOST	D3_SIDE_MISSIONS_LOST	D7_SIDE_MISSIONS_LOST \
	0	0	0
31964	D14_SIDE_MISSIONS_LOST	D21_SIDE_MISSIONS_LOST	D30_SIDE_MISSIONS_LOST \
	0	0	0
31964	D60_SIDE_MISSIONS_LOST	D90_SIDE_MISSIONS_LOST	\
	0	0	
31964	D120_SIDE_MISSIONS_LOST	D180_SIDE_MISSIONS_LOST	\
	0	0	
31964	D240_SIDE_MISSIONS_LOST	D300_SIDE_MISSIONS_LOST	\
	0	0	
31964	D365_SIDE_MISSIONS_LOST	D0_SIDE_MISSIONS_RETREAT	\
	0	0	

31964	D1_SIDE_MISSIONS_RETREAT	0	D3_SIDE_MISSIONS_RETREAT	0	\
31964	D7_SIDE_MISSIONS_RETREAT	0	D14_SIDE_MISSIONS_RETREAT	0	\
31964	D21_SIDE_MISSIONS_RETREAT	0	D30_SIDE_MISSIONS_RETREAT	0	\
31964	D60_SIDE_MISSIONS_RETREAT	0	D90_SIDE_MISSIONS_RETREAT	0	\
31964	D120_SIDE_MISSIONS_RETREAT	0	D180_SIDE_MISSIONS_RETREAT	0	\
31964	D240_SIDE_MISSIONS_RETREAT	0	D300_SIDE_MISSIONS_RETREAT	0	\
31964	D365_SIDE_MISSIONS_RETREAT	0	D0_START_ISIN_CHAPTER	0.0	\
31964	D1_START_ISIN_CHAPTER	NaN	D3_START_ISIN_CHAPTER	0	D7_START_ISIN_CHAPTER \
31964	D14_START_ISIN_CHAPTER	0	D21_START_ISIN_CHAPTER	0	D30_START_ISIN_CHAPTER \
31964	D60_START_ISIN_CHAPTER	0	D90_START_ISIN_CHAPTER	0	\
31964	D120_START_ISIN_CHAPTER	0	D180_START_ISIN_CHAPTER	0	\
31964	D240_START_ISIN_CHAPTER	0	D300_START_ISIN_CHAPTER	0	\
31964	D365_START_ISIN_CHAPTER	0	D0_END_ISIN_CHAPTER	0.0	D1_END_ISIN_CHAPTER \
31964	D3_END_ISIN_CHAPTER	0	D7_END_ISIN_CHAPTER	0	D14_END_ISIN_CHAPTER \
31964	D21_END_ISIN_CHAPTER	0	D30_END_ISIN_CHAPTER	0	D60_END_ISIN_CHAPTER \
31964	D90_END_ISIN_CHAPTER	0	D120_END_ISIN_CHAPTER	0	D180_END_ISIN_CHAPTER \
31964	D240_END_ISIN_CHAPTER	0	D300_END_ISIN_CHAPTER	0	D365_END_ISIN_CHAPTER \
31964	D0_TOTAL_PVP_MATCHES	0	D1_TOTAL_PVP_MATCHES	0	D3_TOTAL_PVP_MATCHES \
31964	D7_TOTAL_PVP_MATCHES	0	D14_TOTAL_PVP_MATCHES	0	D21_TOTAL_PVP_MATCHES \
31964	D30_TOTAL_PVP_MATCHES	0	D60_TOTAL_PVP_MATCHES	0	D90_TOTAL_PVP_MATCHES \
31964	D120_TOTAL_PVP_MATCHES	0	D180_TOTAL_PVP_MATCHES	0	D240_TOTAL_PVP_MATCHES \
31964	D300_TOTAL_PVP_MATCHES	0	D365_TOTAL_PVP_MATCHES	0	\
31964	D0_TOTAL_CAMPAIGN_MISSIONS	0	D1_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D3_TOTAL_CAMPAIGN_MISSIONS	0	D7_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D14_TOTAL_CAMPAIGN_MISSIONS	0	D21_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D30_TOTAL_CAMPAIGN_MISSIONS	0	D60_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D90_TOTAL_CAMPAIGN_MISSIONS	0	D120_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D180_TOTAL_CAMPAIGN_MISSIONS	0	D240_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D300_TOTAL_CAMPAIGN_MISSIONS	0	D365_TOTAL_CAMPAIGN_MISSIONS	0	\
31964	D0_TOTAL_SIDE_MISSIONS	0	D1_TOTAL_SIDE_MISSIONS	0	D3_TOTAL_SIDE_MISSIONS \
31964	D7_TOTAL_SIDE_MISSIONS	0	D14_TOTAL_SIDE_MISSIONS	0	\

```

31964 D21_TOTAL_SIDE_MISSIONS D30_TOTAL_SIDE_MISSIONS \
      0 0
31964 D60_TOTAL_SIDE_MISSIONS D90_TOTAL_SIDE_MISSIONS \
      0 0
31964 D120_TOTAL_SIDE_MISSIONS D180_TOTAL_SIDE_MISSIONS \
      0 0
31964 D240_TOTAL_SIDE_MISSIONS D300_TOTAL_SIDE_MISSIONS \
      0 0
31964 D365_TOTAL_SIDE_MISSIONS D0_CAMPAIGN_WIN_RATE D1_CAMPAIGN_WIN_RATE \
      0 NaN NaN
31964 D3_CAMPAIGN_WIN_RATE D7_CAMPAIGN_WIN_RATE D14_CAMPAIGN_WIN_RATE \
      NaN NaN NaN
31964 D21_CAMPAIGN_WIN_RATE D30_CAMPAIGN_WIN_RATE D60_CAMPAIGN_WIN_RATE \
      NaN NaN NaN
31964 D90_CAMPAIGN_WIN_RATE D120_CAMPAIGN_WIN_RATE D180_CAMPAIGN_WIN_RATE \
      NaN NaN NaN
31964 D240_CAMPAIGN_WIN_RATE D300_CAMPAIGN_WIN_RATE D365_CAMPAIGN_WIN_RATE
      NaN NaN NaN

```

Code cell 20 · saved execution 14

```

a = [2,43]
b= [1,5]
a+b

[2, 43, 1, 5]

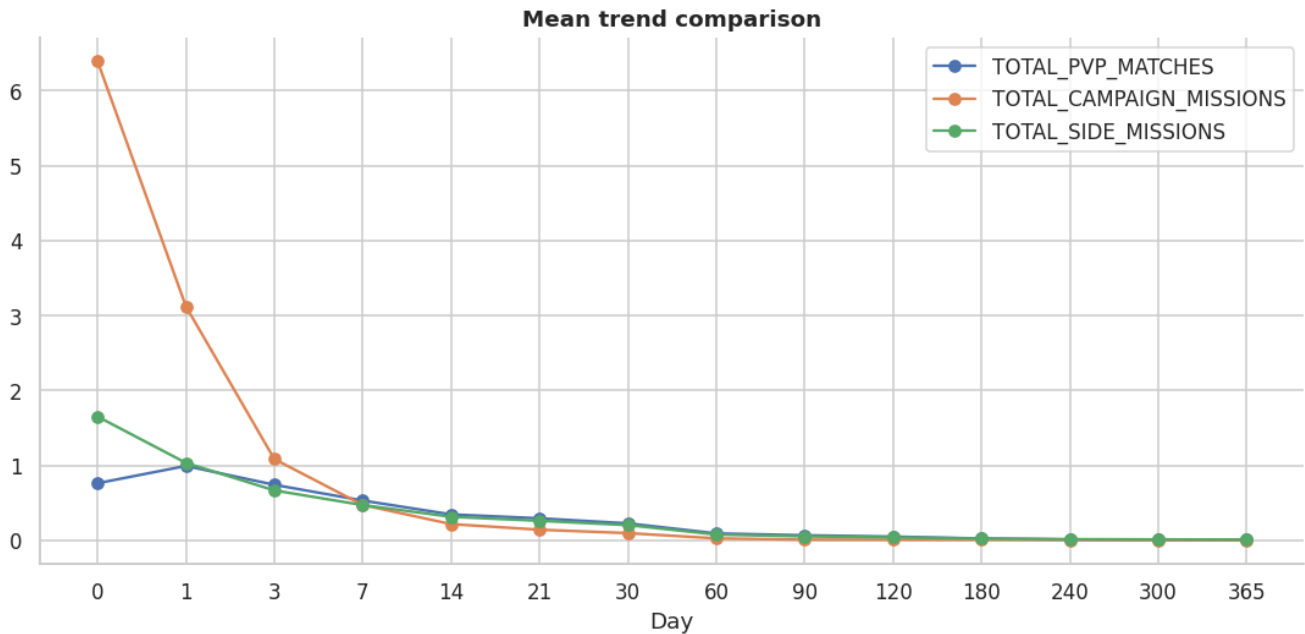
```

Code cell 21 · saved execution 15

```

plot_all_metrics_trend(cleaned_df, ['TOTAL_PVP_MATCHES', 'TOTAL_CAMPAIGN_MISSIONS', 'TOTAL_SIDE_MISSIONS'],
agg='mean')

```



5. Early Spending and Day-One Retention Segments

Code cell 23 · saved execution 16

```

# Early Spender
# players who have spent on D0 or D1 or both days
early_spender = cleaned_df[(cleaned_df['D0_TOTAL_SPEND'] > 0) | (cleaned_df['D1_TOTAL_SPEND'] > 0)]
non_early_spender = cleaned_df[~cleaned_df.index.isin(early_spender.index)]

```

Code cell 24 · saved execution 17

```

## Break segmentation:
# 1. Retained, spender
retained_spender = early_spender[early_spender['D1_TOTAL_SESSION_CT'] > 0]

# 2. Retained, non-spender
retained_non_spender = non_early_spender[non_early_spender['D1_TOTAL_SESSION_CT'] > 0]

# 3. Not retained, spender

```

```

non_retained_spender = early_spender[early_spender['D1_TOTAL_SESSION_CT'] == 0]

# 4. Not retained, non-spender
non_retained_non_spender = non_early_spender[non_early_spender['D1_TOTAL_SESSION_CT'] == 0]

# Segmentation dict
segments = {'Retained Spender': retained_spender,
            'Retained Non-spender': retained_non_spender,
            'Lost Spender': non_retained_spender,
            'Lost Non-spender': non_retained_non_spender}

```

Code cell 25 · saved execution 18

```

def assign_segment(row):
    """Label a player as Retained/Lost x Spender/Non-spender."""
    # early spender = spent on D0 or D1 (NaN > 0 is False, same as your original filter)
    is_spender = (row['D0_TOTAL_SPEND'] > 0) or (row['D1_TOTAL_SPEND'] > 0)

    # D1 retained = at least 1 session on D1
    if row['D1_TOTAL_SESSION_CT'] > 0:
        return 'Retained Spender' if is_spender else 'Retained Non-spender'
    elif row['D1_TOTAL_SESSION_CT'] == 0:
        return 'Lost Spender' if is_spender else 'Lost Non-spender'
    else:
        return np.nan # D1 session count is missing

cleaned_df['segment'] = cleaned_df.apply(assign_segment, axis=1)

cleaned_df['segment'].value_counts(dropna=False)

segment
Lost Non-spender      31787
Retained Non-spender  10387
Retained Spender       102
Lost Spender           20
Name: count, dtype: int64

```

5.1. Segment Sizes and Retention

Our segmentation produces four groups: **Retained Spender (102)**, **Retained Non-spender (10,387)**, **Lost Spender (20)**, and **Lost Non-spender (31,787)**. Early spenders are a fairly small portion of the dataset, with only **122 players spending on D0 or D1**. However, 102 of the 122 early spenders were retained on D1 (about **83.6%**), compared with about **24.6% retention among non-early spenders**. This shows a strong association between early spending and D1 retention, although it does not suggest that spending itself caused retention.

Code cell 27 · saved execution 19

```
seg_order = ['Retained Spender', 'Retained Non-spender', 'Lost Spender', 'Lost Non-spender']
```

5.2. Session Activity

Code cell 29 · saved execution 20

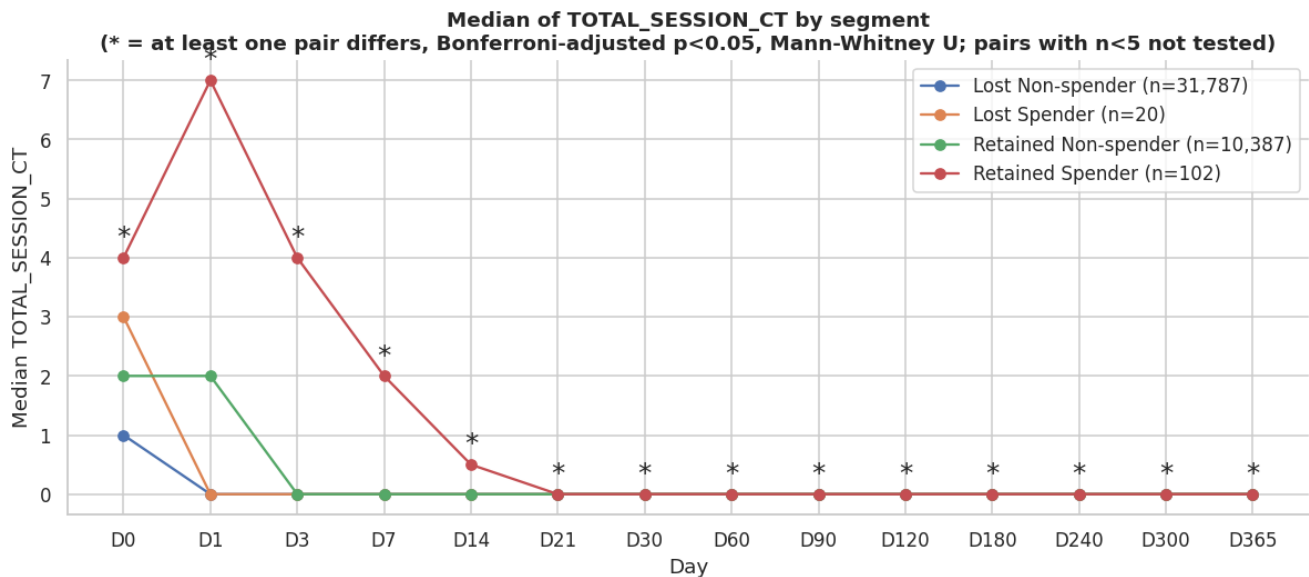
```

# median_df, pair_df = compare_segments_median_test(
#     segments,
#     metric='TOTAL_SESSION_CT',
#     max_day=None,
#     table=False
# )

# # which pairs differ on a given day
# pair_df[(pair_df['day'] == 'D0') & pair_df['significant']]

median_df, pair_df = compare_segments(cleaned_df, 'TOTAL_SESSION_CT', stat='median', table=False)

```



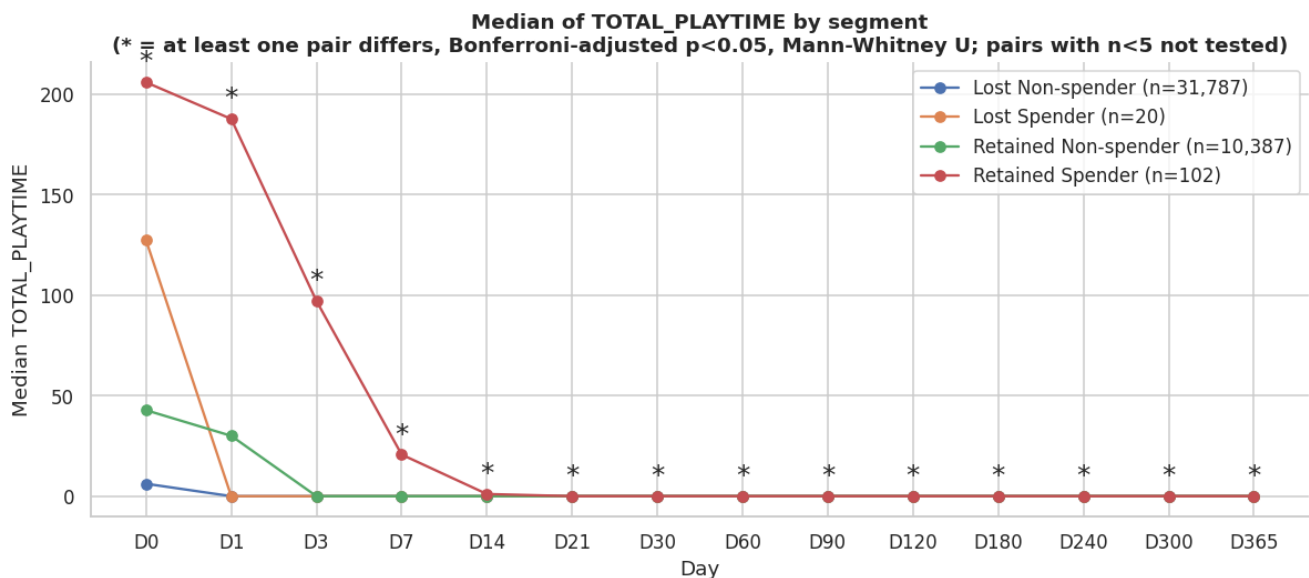
Session Activity: Interpretation

Retained players generally have more game sessions than players who were lost after installation. The retained spender and retained non-spender groups show stronger session activity, while lost players have lower engagement. Returning on D1 seems to be associated with greater early gameplay activity.

5.3. Playtime

Code cell 32 · saved execution 21

```
median_df, pair_df = compare_segments(
    cleaned_df,
    metric='TOTAL_PLAYTIME',
    max_day=None,
    table=False
)
```



Playtime: Interpretation

Playtime follows a similar pattern to sessions. Retained players appear to spend more time playing than lost players, with retained spenders showing especially high engagement. Both the amount of time spent playing and returning on D1 seem to be associated with early player engagement.

5.4. Campaign Win Rates

Code cell 35 · saved execution 22

```

wr_cols = get_day_cols(cleaned_df, 'CAMPAIGN_WIN_RATE') # D0 ... D365, in order
day_labels = [c.split('_')[0] for c in wr_cols]

# 1. Valid players per day, by segment (+ overall)
valid_by_seg = (cleaned_df.groupby('segment')[wr_cols].count().T
                .set_axis(day_labels, axis=0))
valid_by_seg['All'] = cleaned_df[wr_cols].count().values
valid_by_seg = valid_by_seg[seg_order + ['All']] # seg_order from earlier

print('Valid Values for Win Rate for each segment, by day (e.g., Lost Spender did no missions after day 0) ->
all NaN after D0')
valid_by_seg

Valid Values for Win Rate for each segment, by day (e.g., Lost Spender did no missions after day 0) -> all NaN
after D0

```

segment	Retained Spender	Retained Non-spender	Lost Spender	\
D0	99	7108	18	
D1	91	6893	0	
D3	58	2918	0	
D7	40	1722	0	
D14	25	891	0	
D21	19	630	0	
D30	18	408	0	
D60	10	174	0	
D90	5	86	0	
D120	2	55	0	
D180	3	40	0	
D240	0	27	0	
D300	0	22	0	
D365	0	27	0	

segment	Lost Non-spender	All
D0	10523	17748
D1	0	6984
D3	543	3519
D7	313	2075
D14	202	1118
D21	118	767
D30	85	511
D60	33	217
D90	17	108
D120	11	68
D180	8	51
D240	4	31
D300	6	28
D365	3	30

Minimum Sample Sizes for Comparisons

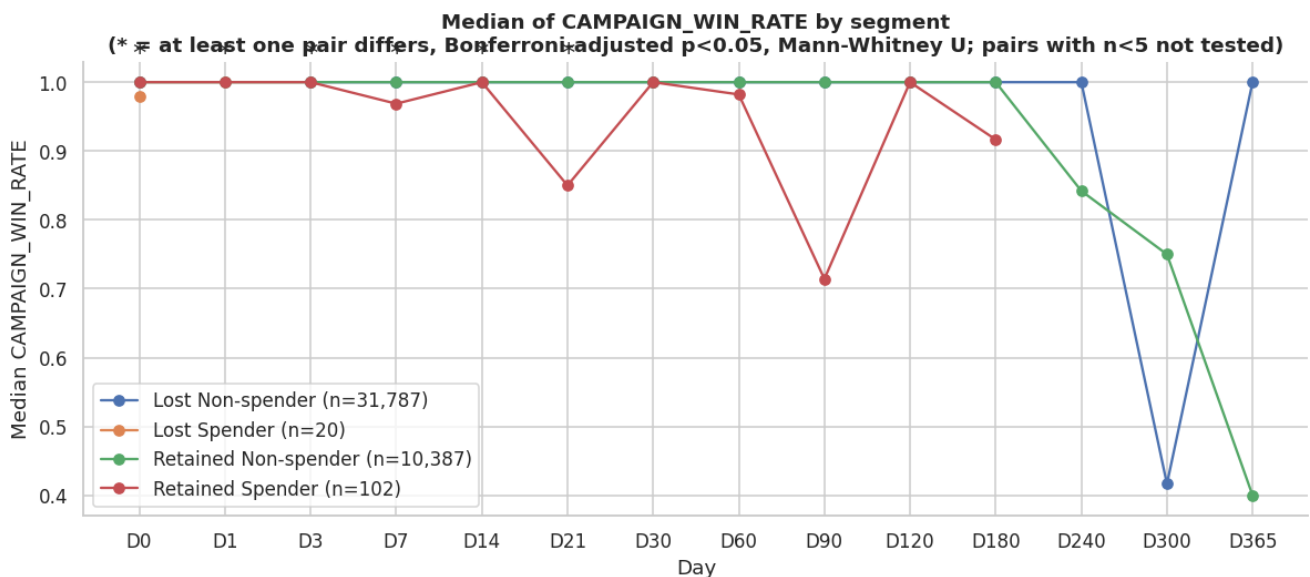
This was evaluated because some groups do not have a large enough sample size for certain days for the t-test. Therefore, to avoid warnings, if either group has fewer than min_n (a parameter inside the **compare_segments**) valid players on a day, the test is skipped and p_value and the effect size were defined as NaN.

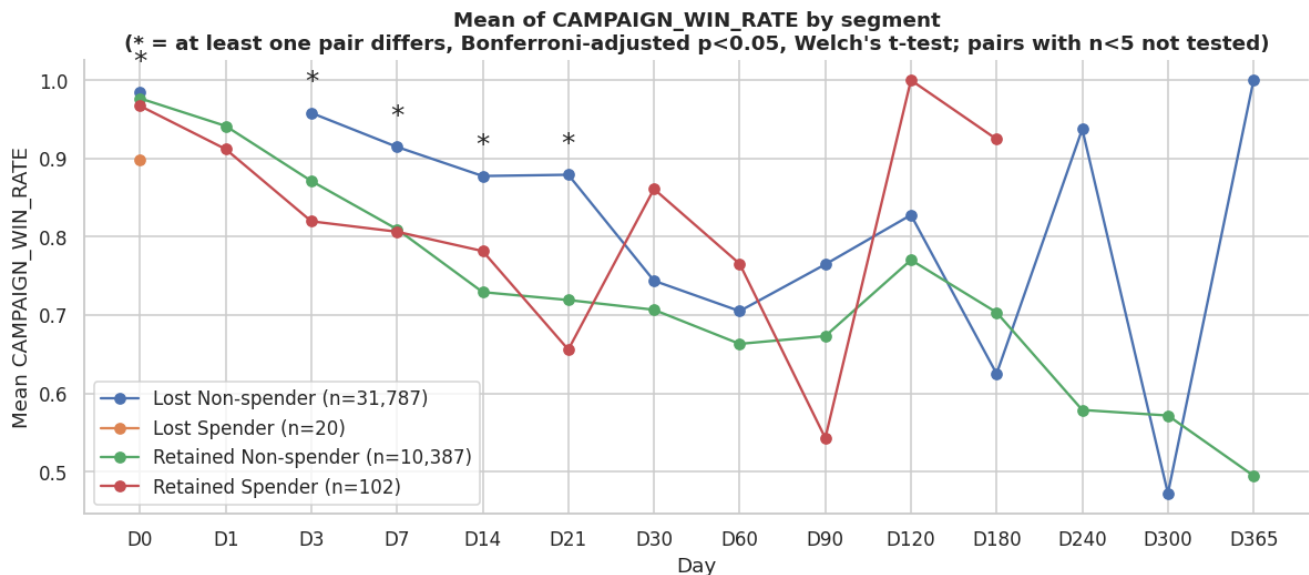
Code cell 37 · saved execution 23

```

for stat in ['median', 'mean']:
    median_df, pair_df = compare_segments(cleaned_df, 'CAMPAIGN_WIN_RATE', stat=stat, table=False, min_n=5)

```





Campaign Win Rates: Interpretation

Overall Campaign Win Rate. Campaign win rates are very high during the earliest days, averaging about **98.1% on D0** and **94.1% on D1**, before overall declining over time. The later-day averages are more skewed as the number of retained players (active players) becomes much smaller.

Win Rate by Segment. The retained groups continue to have valid campaign activity spread across more days because they return to the game, while the lost groups show fewer later observations. In particular, the **Lost Spender group has no valid post-D0 observations** because these players did not return on D1. Differences in later-day win rates between segments should not be interpreted without considering the different sample sizes.

5.5. Campaign Chapter Progression

Code cell 40 · saved execution 24

```
def add_max_chapter_reached(df):
    """Add D{d}_MAX_CHAPTER_REACHED = furthest chapter reached up to and including day d."""
    end_cols = get_day_cols(df, 'END_ISIN_CHAPTER')
    mx = df[end_cols].fillna(0).cummax(axis=1)
    mx.columns = [c.replace('END_ISIN_CHAPTER', 'MAX_CHAPTER_REACHED') for c in end_cols]
    return pd.concat([df.drop(columns=mx.columns, errors='ignore'), mx], axis=1)
```

```
cleaned_df = add_max_chapter_reached(cleaned_df)
```

Code cell 41 · saved execution 25

```
def plot_chapter_reach(df, day='D30', segment_col='segment', segment_order=None,
                      max_chapter=8, alpha=0.05):
    col = f'{day}_MAX_CHAPTER_REACHED'
    order = list(segment_order) if segment_order is not None \
        else sorted(df[segment_col].dropna().unique())
    ks = np.arange(1, max_chapter + 1)

    vals = {s: df.loc[df[segment_col] == s, col].dropna() for s in order}
    reach = pd.DataFrame({s: [(v >= k).mean() * 100 for k in ks] for s, v in vals.items()},
                        index=[f'>= Chap {k}' for k in ks]).round(1)

    fig, ax = plt.subplots(figsize=(9, 5))
    for s, v in vals.items():
        ax.plot(ks, reach[s].values, marker='o', label=f'{s} (n={len(v):})')
    ax.set_xticks(ks)
    ax.set_xlabel('Chapter'); ax.set_ylabel('% of players who reached at least this chapter')
    ax.set_title(f'Percentage of player reaching that chapter by {day}')
    ax.legend(); plt.tight_layout(); plt.show()

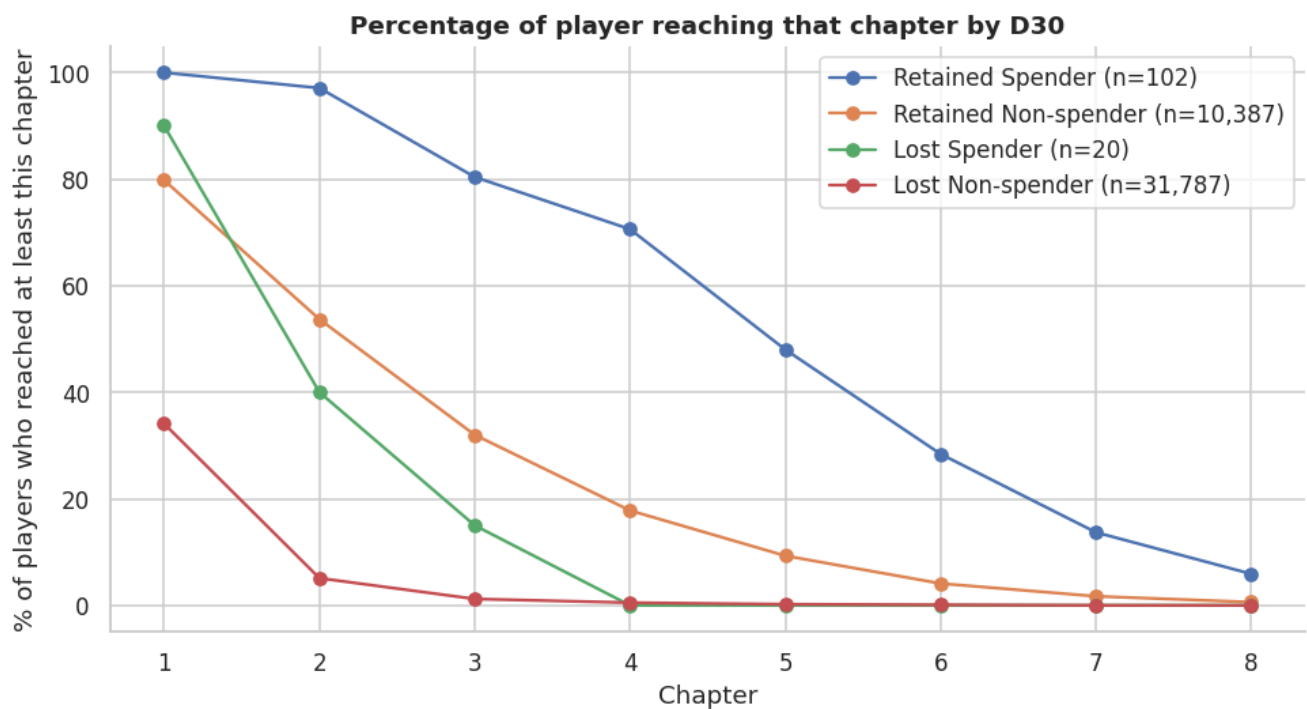
# one Mann-Whitney per pair on the furthest chapter reached (Bonferroni-adjusted)
pairs = list(combinations(order, 2))
rows = []
for a, b in pairs:
    u, p = stats.mannwhitneyu(vals[a], vals[b], alternative='two-sided', method='asymptotic')
    rows.append({'group_a': a, 'group_b': b,
                'mean_a': vals[a].mean(), 'mean_b': vals[b].mean(),
                'p_adj': min(p * len(pairs), 1.0),
                'rank_biserial': 2 * u / (len(vals[a]) * len(vals[b])) - 1})
test_df = pd.DataFrame(rows)
test_df['significant'] = test_df['p_adj'] < alpha
```

```

return reach, test_df

# day are adjustable
reach_tbl, test_df = plot_chapter_reach(cleaned_df, day='D30', segment_order=seg_order)
reach_tbl

```



```

>= Chap 1      Retained Spender      Retained Non-spender      Lost Spender      \
>= Chap 2      100.0                  79.8                  90.0
>= Chap 3      97.1                  53.7                  40.0
>= Chap 4      80.4                  32.0                  15.0
>= Chap 5      70.6                  17.8                   0.0
>= Chap 6      48.0                   9.3                   0.0
>= Chap 7      28.4                   4.1                   0.0
>= Chap 8      13.7                   1.7                   0.0
>= Chap 9       5.9                   0.6                   0.0

>= Chap 1      Lost Non-spender
>= Chap 2       5.1
>= Chap 3       1.2
>= Chap 4       0.5
>= Chap 5       0.2
>= Chap 6       0.1
>= Chap 7       0.0
>= Chap 8       0.0

```

Chapter Progression: Interpretation

The chapter results show a clear difference in progression between retained and lost players. By D30, **97.1% of retained spenders had reached at least Chapter 2 and 80.4% had reached at least Chapter 3**, compared with **53.7% and 32.0% for retained non-spenders**. Lost non-spenders show less progression, with only **5.1% reaching at least Chapter 2**. Players who remain active and return on D1 generally progress much farther through the Campaign.

Code cell 43 · saved execution 26

```

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42296 entries, 0 to 42295
Data columns (total 22 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Player ID                            42296 non-null  object
 1   INSTL_TS_UTC                         42296 non-null  object
 2   AD_NETWORK                           41794 non-null  object
 3   AD_CHANNEL                           42296 non-null  object
 4   COUNTRY                             42294 non-null  object
 5   CITY                                40559 non-null  object
 6   LANGUAGE                             42296 non-null  object
 7   DEVICE_BRAND                         42296 non-null  object
 8   DX_TOTAL_SESSION_CT                 42296 non-null  object
 9   DX_TOTAL_PLAYTIME                   42296 non-null  object
10  DX_TOTAL_SPEND                       42296 non-null  object
11  DX_CAMPAIGN_MISSIONS_WON            42296 non-null  object

```

```

12 DX_CAMPAIGN_MISSIONS_LOST      42296 non-null object
13 DX_CAMPAIGN_MISSIONS_RETREAT   42296 non-null object
14 DX_PVP_MATCHES_WON             42296 non-null object
15 DX_PVP_MATCHES_LOST           42296 non-null object
16 DX_PVP_MATCHES_RETREAT        42296 non-null object
17 DX_SIDE_MISSIONS_WON           42296 non-null object
18 DX_SIDE_MISSIONS_LOST         42296 non-null object
19 DX_SIDE_MISSIONS_RETREAT      42296 non-null object
20 DX_START_ISIN_CHAPTER         42296 non-null object
21 DX_END_ISIN_CHAPTER           42296 non-null object
dtypes: object(22)
memory usage: 7.1+ MB

```

6. Behavioral Clustering

6.1. Feature Selection and Preprocessing

Code cell 45 · saved execution 27

```

from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Isolate metrics for clustering for D0
metrics = ['TOTAL_PLAYTIME', 'TOTAL_SESSION_CT', 'TOTAL_SPEND',
           'TOTAL_PVP_MATCHES', 'TOTAL_CAMPAIGN_MISSIONS', 'TOTAL_SIDE_MISSIONS', 'END_ISIN_CHAPTER']
feat_cols = [f'D0_{m}' for m in metrics] # D0 only

# Set new dataframe
clus_df = cleaned_df[feat_cols].copy()

```

Code cell 46 · saved execution 28

```

pip install factor_analyzer

Collecting factor_analyzer
  Downloading factor_analyzer-0.5.1.tar.gz (42 kB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0m ━━━━━━━━━━ 32m0.0/42.8 kB ━━━━━━━━━━ 0m ━━━━━━━━━━ 31m? ━━━━━━━━━━ 0m eta
    ━━━━━━━━━━━ 36m-:--:-- ━━━━━━━━━━ 0m
    ━━━━━━━━━━━ 2K ━━━━━━━━━━━ 90m ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0m ━━━━━━━━━━ 32m42.8/42.8 kB ━━━━━━━━━━ 0m ━━━━━━━━━━ 31m1.9 MB/s ━━━━━━━━━━ 0m eta
    ━━━━━━━━━━━ 36m0:00:00 ━━━━━━━━━━ 0m
    ━━━━━━━━━━━ 25h Installing build dependencies ... ━━━━━━━━━━━ 25l ━━━━━━━━━━━ 25hdone
      Getting requirements to build wheel ... ━━━━━━━━━━━ 25l ━━━━━━━━━━━ 25hdone
      Preparing metadata (pyproject.toml) ... ━━━━━━━━━━━ 25l ━━━━━━━━━━━ 25hdone
Requirement already satisfied: pandas in /usr/local/lib/python3.13/dist-packages (from factor_analyzer) (2.2.3)
Requirement already satisfied: scipy in /usr/local/lib/python3.13/dist-packages (from factor_analyzer) (1.16.3)
Requirement already satisfied: numpy in /usr/local/lib/python3.13/dist-packages (from factor_analyzer) (2.1.3)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.13/dist-packages (from factor_analyzer) (1.6.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from
pandas->factor_analyzer) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from
pandas->factor_analyzer) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from
pandas->factor_analyzer) (2026.4)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from scikit-
learn->factor_analyzer) (1.6.0)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-
learn->factor_analyzer) (3.7.0)
Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from
joblib>=1.2.0->scikit-learn->factor_analyzer) (3.1.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-
dateutil>=2.8.2->pandas->factor_analyzer) (1.17.0)
Building wheels for collected packages: factor_analyzer
  Building wheel for factor_analyzer (pyproject.toml) ... ━━━━━━━━━━━ 25l ━━━━━━━━━━━ 25hdone
    Created wheel for factor_analyzer: filename=factor_analyzer-0.5.1-py2.py3-none-any.whl size=42655
    sha256=2080967fb728766083b0ee979f988a92c9ccbbfc8e99e27cef9294bba00de97f
    Stored in directory: /root/.cache/pip/wheels/87/6e/8f/914f20e0242ee0214e5c8336031a2fab12e632e4695fbb7276
Successfully built factor_analyzer
Installing collected packages: factor_analyzer
Successfully installed factor_analyzer-0.5.1

```

Code cell 47 · saved execution 29

```

X = clus_df.fillna(0)
X = np.log1p(X) # tame skewed data (assumes no negatives)
X_scaled = StandardScaler().fit_transform(X)
#Assess correlation between features
corr=X.corr()
print(corr)

sns.heatmap(corr, cmap=sns.diverging_palette(20, 220, n=200))

from factor_analyzer.factor_analyzer import calculate_kmo, calculate_bartlett_sphericity

#KMO
kmo_all_vars, kmo_overall=calculate_kmo(X_scaled)
print(kmo_overall)

#Bartlett's Test

```

```
chi_square_value, p_value=calculate_bartlett_sphericity(X_scaled)
print(p_value)
```

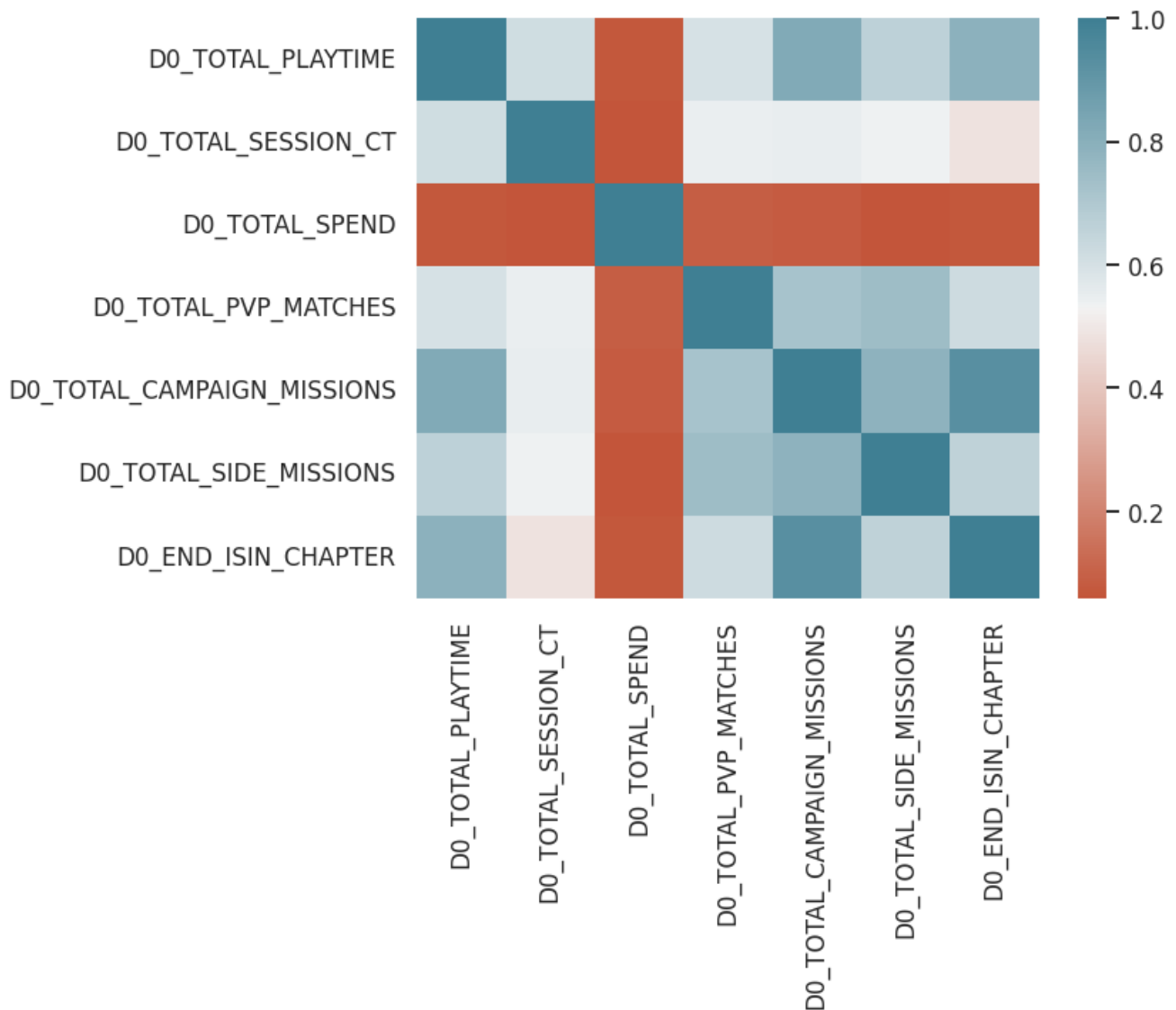
	D0_TOTAL_PLAYTIME	D0_TOTAL_SESSION_CT	\
D0_TOTAL_PLAYTIME	1.000000	0.616809	
D0_TOTAL_SESSION_CT	0.616809	1.000000	
D0_TOTAL_SPEND	0.071507	0.058889	
D0_TOTAL_PVP_MATCHES	0.598768	0.543589	
D0_TOTAL_CAMPAIGN_MISSIONS	0.818755	0.549937	
D0_TOTAL_SIDE_MISSIONS	0.664122	0.537321	
D0_END_ISIN_CHAPTER	0.791385	0.486454	

	D0_TOTAL_SPEND	D0_TOTAL_PVP_MATCHES	\
D0_TOTAL_PLAYTIME	0.071507	0.598768	
D0_TOTAL_SESSION_CT	0.058889	0.543589	
D0_TOTAL_SPEND	1.000000	0.089044	
D0_TOTAL_PVP_MATCHES	0.089044	1.000000	
D0_TOTAL_CAMPAIGN_MISSIONS	0.079949	0.720335	
D0_TOTAL_SIDE_MISSIONS	0.063390	0.749020	
D0_END_ISIN_CHAPTER	0.070265	0.619547	

	D0_TOTAL_CAMPAIGN_MISSIONS	\
D0_TOTAL_PLAYTIME	0.818755	
D0_TOTAL_SESSION_CT	0.549937	
D0_TOTAL_SPEND	0.079949	
D0_TOTAL_PVP_MATCHES	0.720335	
D0_TOTAL_CAMPAIGN_MISSIONS	1.000000	
D0_TOTAL_SIDE_MISSIONS	0.785505	
D0_END_ISIN_CHAPTER	0.933064	

	D0_TOTAL_SIDE_MISSIONS	D0_END_ISIN_CHAPTER
D0_TOTAL_PLAYTIME	0.664122	0.791385
D0_TOTAL_SESSION_CT	0.537321	0.486454
D0_TOTAL_SPEND	0.063390	0.070265
D0_TOTAL_PVP_MATCHES	0.749020	0.619547
D0_TOTAL_CAMPAIGN_MISSIONS	0.785505	0.933064
D0_TOTAL_SIDE_MISSIONS	1.000000	0.661181
D0_END_ISIN_CHAPTER	0.661181	1.000000

0.8375155643555982
0.0



Feature Selection: Interpretation

The clustering analysis uses early (D0) player behavior including **playtime, session count, spend, total Campaign Missions, total PVP Matches, total Side Missions, and the chapter the player ended in**. The variables were log-transformed (log1p) and standardized so that highly skewed measures, especially spending and playtime, would not dominate the clustering. Checked KMO and Bartlett results to determine whether this is a valid dataset to perform clustering.

6.2. K-Means: Model Selection

Code cell 50 · saved execution 30

```
#Calculate silhouette score
for k in range(2,9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled)
    sil = silhouette_score(X_scaled, km.labels_, sample_size=10000, random_state=42)
    print(k, round(sil, 3))

2 0.603
3 0.524
4 0.529
5 0.516
6 0.479
7 0.48
8 0.48
```

Code cell 51 · saved execution 31

```
k_range = range(2, 10)
inertias, sils = [], []

for k in k_range:
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled)
    inertias.append(km.inertia_)
```

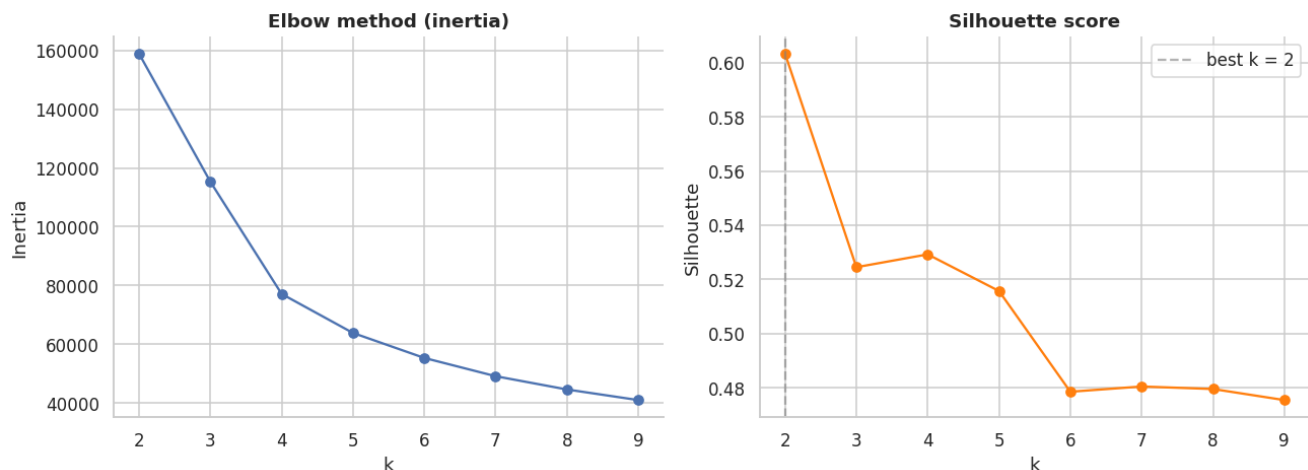
```

sils.append(silhouette_score(X_scaled, km.labels_, sample_size=10000, random_state=42))

fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
#Plot elbow method
ax[0].plot(k_range, inertias, marker='o')
ax[0].set_title('Elbow method (inertia)')
ax[0].set_xlabel('k')
ax[0].set_ylabel('Inertia')
ax[0].set_xticks(list(k_range))
#Plot silhouette score
ax[1].plot(k_range, sils, marker='o', color='tab:orange')
best_k = list(k_range)[int(np.argmax(sils))]
ax[1].axvline(best_k, ls='--', color='gray', alpha=0.6, label=f'best k = {best_k}')
ax[1].set_title('Silhouette score')
ax[1].set_xlabel('k')
ax[1].set_ylabel('Silhouette')
ax[1].set_xticks(list(k_range))
ax[1].legend()

plt.tight_layout()
plt.show()

```



Choosing the Number of Clusters

The silhouette results were **0.612 for K=2**, **0.609 for K=3**, and **0.473 for K=4**, with lower scores for larger values of K. Since a higher silhouette score indicates better separation, K=2 has the highest numerical score. However, **K=4 was selected** to provide more detailed and interpretable player segments. Additionally, K=4 was chosen for behavioral detail because it had the highest silhouette score after K=2 and is where the elbow graph changes slope.

Code cell 53 · saved execution 32

```

# fit final model
km = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X_scaled)
cleaned_df['cluster'] = km.labels_

```

6.3. K-Means: Cluster Profiles

Code cell 55 · saved execution 33

```
cleaned_df.groupby('cluster')[feat_cols].mean() # profile each cluster
```

	D0_TOTAL_PLAYTIME	D0_TOTAL_SESSION_CT	D0_TOTAL_SPEND	\
cluster				
0	5.915956	1.301280	0.000288	
1	159.337427	4.791195	0.007229	
2	41.593760	1.772460	0.000227	
3	218.393091	4.509091	127.950909	

	D0_TOTAL_PVP_MATCHES	D0_TOTAL_CAMPAIGN_MISSIONS	\
cluster			
0	0.000207	0.001033	
1	6.177908	38.439620	
2	0.096567	5.867650	
3	8.418182	59.800000	

	D0_TOTAL_SIDE_MISSIONS	D0_END_ISIN_CHAPTER
cluster		
0	0.000083	0.000000
1	11.596123	1.971123
2	0.910849	1.022368
3	10.745455	2.296296

Code cell 56 · saved execution 34

```
cleaned_df.groupby('cluster')['D1_TOTAL_SESSION_CT'].apply(lambda s: (s > 0).mean()) # D1 retention rate

cluster
0    0.126311
1    0.742326
2    0.284043
3    0.727273
Name: D1_TOTAL_SESSION_CT, dtype: float64
```

Day-One Retention by Cluster

D1 retention differs greatly between the four clusters. **Cluster 2 has the lowest D1 retention at 11.9%**, while **Cluster 1 has 75.8% retention** and **Cluster 3 has 72.7%**. Cluster 0 falls between them at **33.9%**. The clustering captures meaningful differences in early engagement and D1 retention, with the lowest-activity cluster also showing the lowest retention.

Code cell 58 · saved execution 35

```
from matplotlib.ticker import PercentFormatter

cat_cols = ['AD_NETWORK', 'AD_CHANNEL', 'COUNTRY', 'CITY', 'LANGUAGE', 'DEVICE_BRAND']
top_n = 8

clusters = sorted(cleaned_df['cluster'].unique())
colors = [plt.cm.tab10(i) for i in range(len(clusters))]

# overall cluster mix -> cumulative boundaries used as reference lines
overall_cum = cleaned_df['cluster'].value_counts(normalize=True).sort_index().cumsum()

fig, ax = plt.subplots(2, 3, figsize=(20, 10))
ax = ax.ravel()

for i, c in enumerate(cat_cols):
    s = cleaned_df[c].fillna('Unknown').astype(str)
    s = s.where(s.isin(s.value_counts().index[:top_n]), 'Other')

    # category order: largest first, "Other" last
    order = s.value_counts().index.tolist()
    if 'Other' in order:
        order = [o for o in order if o != 'Other'] + ['Other']

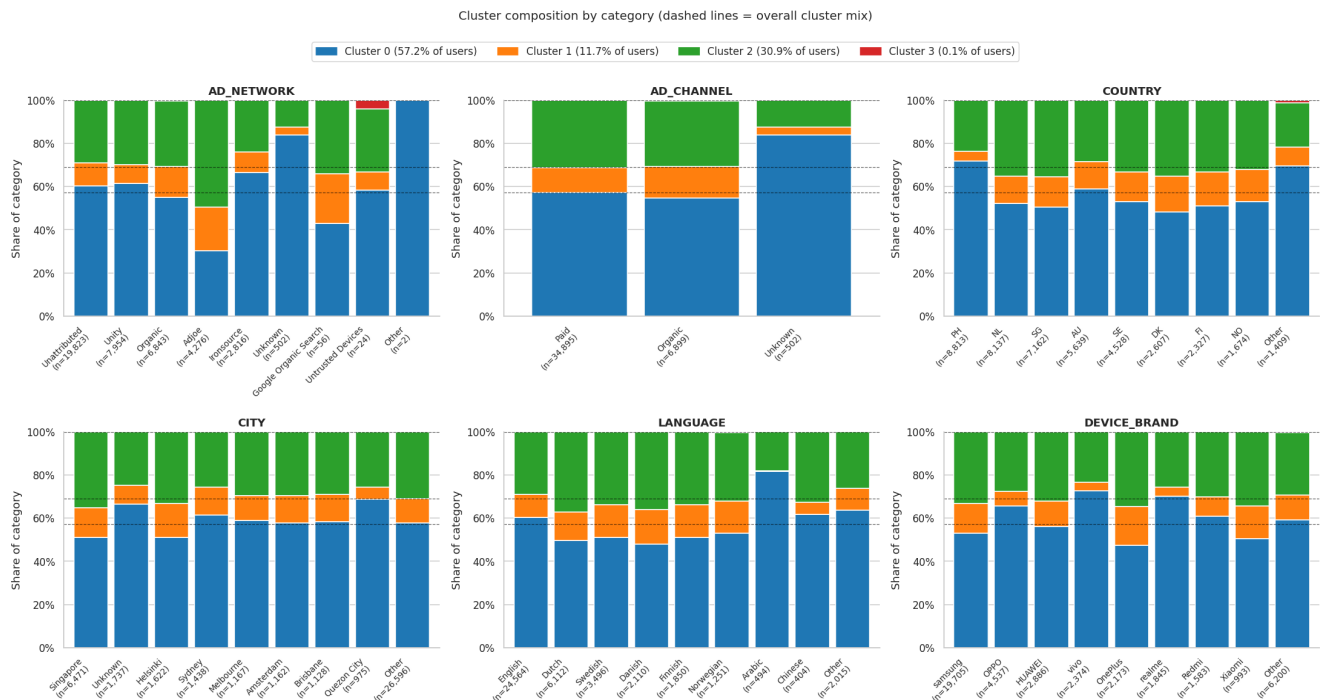
    # row-normalized: each category (bar) sums to 100% across clusters
    ct = pd.crosstab(s, cleaned_df['cluster'], normalize='index').reindex(order)
    n_per_cat = s.value_counts().reindex(order)

    ct.plot(kind='bar', stacked=True, ax=ax[i], width=0.85, legend=False, color=colors)

    # reference lines: overall cluster mix
    for y in overall_cum.iloc[:-1]:
        ax[i].axhline(y, ls='--', color='black', lw=0.8, alpha=0.6)

    ax[i].set_ylim(0, 1)
    ax[i].yaxis.set_major_formatter(PercentFormatter(1.0))
    ax[i].set_xticklabels([f'{o}\n(n={n:,})' for o, n in zip(order, n_per_cat)],
                          rotation=45, ha='right', fontsize=9)
    ax[i].set_title(c)
    ax[i].set_xlabel('')
    ax[i].set_ylabel('Share of category')

# one shared legend
handles = [plt.Rectangle((0, 0), 1, 1, color=colors[k]) for k in range(len(clusters))]
share = cleaned_df['cluster'].value_counts(normalize=True).sort_index()
fig.legend(handles, [f'Cluster {k} ({share[k]:.1%} of users)' for k in clusters],
           loc='upper center', ncol=len(clusters), bbox_to_anchor=(0.5, 1.02))
fig.suptitle('Cluster composition by category (dashed lines = overall cluster mix)', y=1.06, fontsize=13)
plt.tight_layout()
plt.show()
```



Acquisition and Demographic Profiles

These charts show how each category is split across the four behavioral clusters, with dashed lines marking the overall cluster mix. They only describe cluster composition and do not establish that any category caused higher or lower engagement.

- **Acquisition Network:** shows whether particular advertising networks or organic sources are over- or under-represented in certain behavior groups.
- **Acquisition Channel:** shows the proportion of **Paid**, **Organic**, and **Unknown** players in each cluster. Since Paid makes up most of the dataset, it is present across the major clusters; the comparison shows whether more and less engaged clusters have similar acquisition mixes.
- **Country:** provides geographic context for the segments and shows whether certain countries are relatively more represented in particular clusters. This is merely descriptive and not evidence that country determines engagement.
- **City:** a more detailed geographic breakdown. Singapore is a large part of the dataset, while Helsinki, Sydney, Melbourne, and Amsterdam also appear among the major cities.
- **Language:** English is highest overall, so it makes up a large portion of the major clusters. Smaller language groups have smaller sample sizes, so their differences should be interpreted carefully.
- **Device Brand:** Samsung is the largest brand overall, so it represents a substantial portion of the player base across the clusters. This comparison shows whether device-brand composition differs between behavioral segments.

Code cell 60 · saved execution 36

```
cleaned_df['D1_RETAINED'] = (cleaned_df['D1_TOTAL_SESSION_CT'] > 0).astype(int)
plot_cols = feat_cols + ['D1_RETAINED']

clusters = sorted(cleaned_df['cluster'].unique())
colors = [plt.cm.tab10(i) for i in range(len(clusters))] # same colors as the stacked bars
sizes = cleaned_df['cluster'].value_counts().sort_index()

# ----- 1. Mean per cluster (± SEM), one panel per metric -----
# Changed grid size from (2, 3) to (3, 3) to support 8 metrics
fig, ax = plt.subplots(3, 3, figsize=(18, 12))
ax = ax.ravel()

for i, col in enumerate(plot_cols):
    g = cleaned_df.groupby('cluster')[col]
    means, sems = g.mean(), g.sem()

    bars = ax[i].bar([str(k) for k in clusters], means.values, yerr=sems.values,
                     capsize=4, color=colors)
    ax[i].bar_label(bars, fmt='%0.2f' if col == 'D1_RETAINED' else '%.1f',
                   padding=3, fontsize=9)
    ax[i].set_title(col + (' (retention rate)' if col == 'D1_RETAINED' else ''))
    ax[i].set_xlabel('Cluster')
    ax[i].set_ylabel('Mean')
    ax[i].set_xticks(range(len(clusters)))
    ax[i].set_xticklabels([f'{k}\n(n={sizes[k]:,})' for k in clusters])
    ax[i].set_ylim(0, (means + sems).max() * 1.2)

# Hide any unused subplots (the 9th subplot at index 8)
```

```

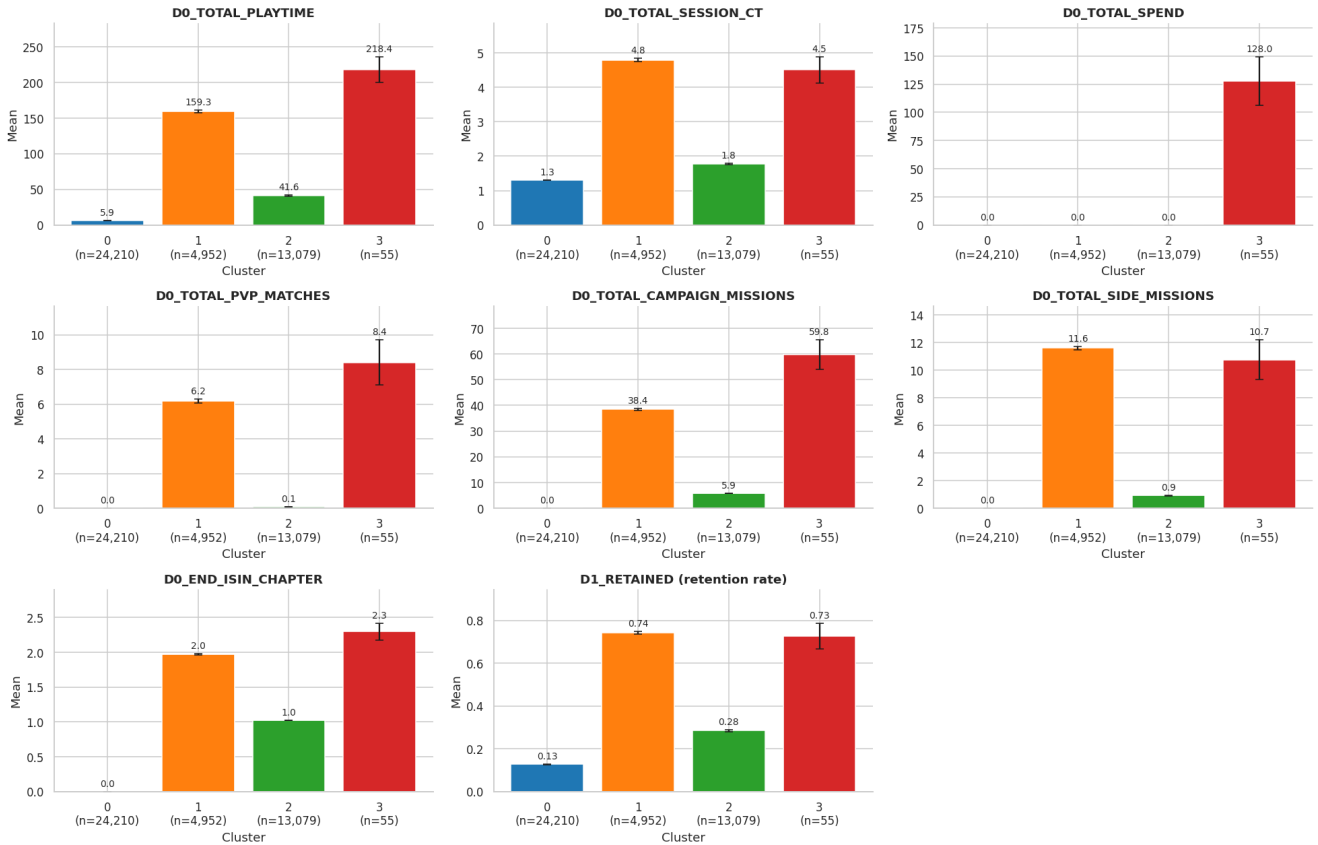
for j in range(len(plot_cols), len(ax)):
    ax[j].axis('off')

fig.suptitle('Cluster comparison across key metrics (mean ± SEM)', y=1.02, fontsize=13)
plt.tight_layout()
plt.show()

# ----- 2. Medians, since the means are skewed -----
display(cleaned_df.groupby('cluster')[plot_cols].median())

```

Cluster comparison across key metrics (mean ± SEM)



	D0_TOTAL_PLAYTIME	D0_TOTAL_SESSION_CT	D0_TOTAL_SPEND	\
cluster				
0	3.670	1.0	0.00	
1	122.555	4.0	0.00	
2	22.830	1.0	0.00	
3	211.180	4.0	69.97	

	D0_TOTAL_PVP_MATCHES	D0_TOTAL_CAMPAIGN_MISSIONS	\
cluster			
0	0.0	0.0	
1	3.0	31.0	
2	0.0	4.0	
3	4.0	58.0	

	D0_TOTAL_SIDE_MISSIONS	D0_END_ISIN_CHAPTER	D1_RETAINED
cluster			
0	0.0	0.0	0.0
1	9.0	2.0	1.0
2	0.0	1.0	0.0
3	8.0	2.0	1.0

Quantitative Engagement Profiles

The four clusters represent distinct levels of early engagement. **Cluster 2**, the largest group with 25,837 players, has very low activity, averaging only **5.08 minutes of playtime** and **1.21 sessions**. **Cluster 0** is a moderately active but mostly non-spending group, while **Cluster 1** shows substantially higher playtime, sessions, Campaign activity, and PVP activity. **Cluster 3**, which only has 55 players, has the highest average spending and high gameplay activity, representing a small but high-value player group.

6.4. Country and Language Distributions

Now that K-Means clusters have been identified based on numerical activity metrics, let's examine the distribution of categorical features (top 5 countries and languages) within these clusters. This will help characterize each cluster further.

Code cell 63 · saved execution 37

```
from sklearn.preprocessing import OneHotEncoder
from matplotlib.ticker import PercentFormatter
import matplotlib.pyplot as plt

# Identify top 5 countries and languages based on frequency
top_countries = cleaned_df['COUNTRY'].value_counts().nlargest(5).index.tolist()
top_languages = cleaned_df['LANGUAGE'].value_counts().nlargest(5).index.tolist()

# Create new categorical columns in cleaned_df that group less frequent categories into 'Other'
cleaned_df['COUNTRY_CAT'] = cleaned_df['COUNTRY'].apply(lambda x: x if x in top_countries else 'Other')
cleaned_df['LANGUAGE_CAT'] = cleaned_df['LANGUAGE'].apply(lambda x: x if x in top_languages else 'Other')

# Define the categorical columns for plotting
cat_ohe_plot_cols = ['COUNTRY_CAT', 'LANGUAGE_CAT']

clusters = sorted(cleaned_df['cluster'].unique())

# Get all unique categories across both features for consistent color mapping
all_unique_categories_for_legend = sorted(list(set(top_countries + top_languages + ['Other'])))
# Ensure 'Other' is last in the legend if present
all_unique_categories_for_legend.sort(key=lambda x: (x == 'Other', x))

# New color map for categories for stacking (up to the total number of unique categories)
# Using plt.colormaps.get_cmap to avoid deprecation warning
category_cmap = plt.colormaps.get_cmap('tab10')
category_colors = [category_cmap(i) for i in range(len(all_unique_categories_for_legend))]

# Create a mapping from category name to its assigned color for consistency
category_color_map = {cat: category_colors[i] for i, cat in enumerate(all_unique_categories_for_legend)}

fig, ax = plt.subplots(1, 2, figsize=(18, 7))
ax = ax.ravel()

for i, c in enumerate(cat_ohe_plot_cols):
    s = cleaned_df[c].fillna('Unknown').astype(str)

    # Determine category order for stacking to ensure consistency
    current_categories_in_s = s.value_counts().index.tolist()
    category_order_for_plot = []
    if c == 'COUNTRY_CAT':
        # Filter top_countries to only include those present in the current column's data
        category_order_for_plot = [x for x in top_countries if x in current_categories_in_s]
    elif c == 'LANGUAGE_CAT':
        # Filter top_languages to only include those present in the current column's data
        category_order_for_plot = [x for x in top_languages if x in current_categories_in_s]

    # Add 'Other' if it's present in the current column's data, ensuring it's last
    if 'Other' in current_categories_in_s and 'Other' not in category_order_for_plot:
        category_order_for_plot.append('Other')

    # Ensure consistent ordering for plotting by re-ordering based on the full list if needed
    # This also helps pick colors consistently
    category_order_for_plot_filtered = [cat for cat in all_unique_categories_for_legend if cat in category_order_for_plot]

    # Calculate proportions of each category WITHIN each cluster
    # Index = clusters, Columns = categories. Normalize by index (cluster) so each bar sums to 1.
    ct = pd.crosstab(cleaned_df['cluster'], s, normalize='index')
    # Reindex columns to ensure order and fill missing categories, and apply color mapping
    ct = ct.reindex(columns=category_order_for_plot_filtered, fill_value=0)

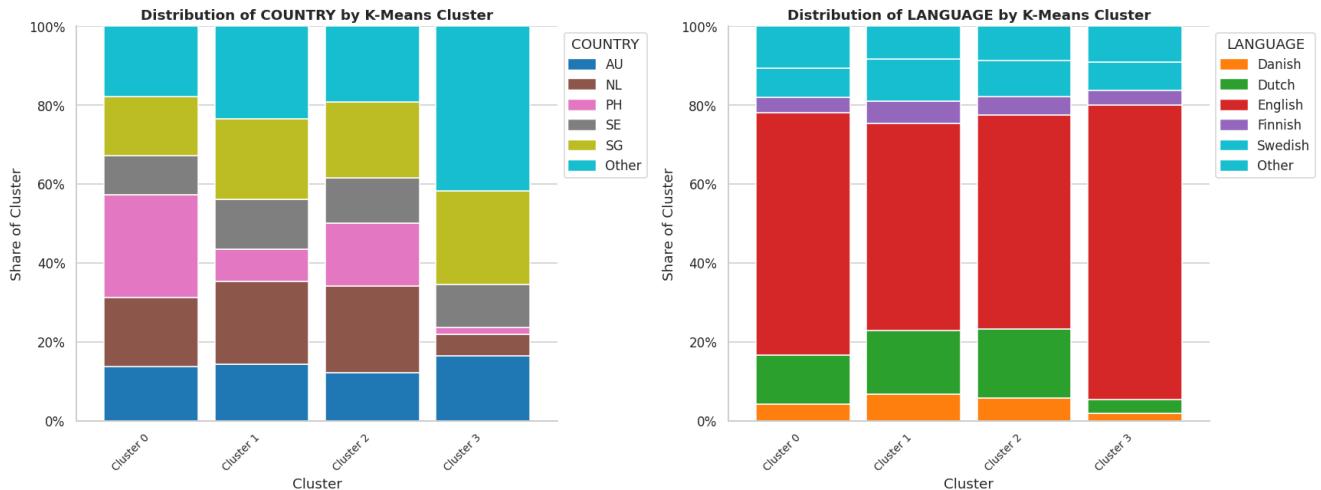
    # Get colors for the categories actually plotted in this subplot
    colors_for_this_plot = [category_color_map[cat] for cat in ct.columns]

    # Plot stacked bar chart
    ct.plot(kind='bar', stacked=True, ax=ax[i], width=0.85, legend=False, color=colors_for_this_plot)

    ax[i].set_ylim(0, 1)
    ax[i].yaxis.set_major_formatter(PercentFormatter(1.0))
    # X-axis labels are now clusters
    ax[i].set_xticklabels([f'Cluster {k}' for k in clusters], rotation=45, ha='right', fontsize=9)
    ax[i].set_title(f'Distribution of {c.replace("_CAT", "")} by K-Means Cluster')
    ax[i].set_xlabel('Cluster')
    ax[i].set_ylabel('Share of Cluster')

    # Create individual legend for each subplot
    present_categories_for_subplot = [cat for cat in category_order_for_plot_filtered if cat in ct.columns]
    subplot_legend_handles = [plt.Rectangle((0, 0), 1, 1, color=category_color_map[cat]) for cat in present_categories_for_subplot]
    ax[i].legend(subplot_legend_handles, present_categories_for_subplot,
        loc='upper left', bbox_to_anchor=(1, 1), title=c.replace('_CAT', ''))

# Adjust title and layout
fig.suptitle('K-Means Cluster Composition by Categorical Features', y=1.02, fontsize=13)
plt.tight_layout(rect=[0, 0.03, 0.88, 0.95]) # Adjust layout to make space for legends on the right
plt.show()
```



Geographic and Language Patterns: Interpretation

These plots show how the top countries and languages are distributed within each of the K-Means clusters. 1. High-Value "Whales" (Cluster 3) are Dominantly English-Speaking

Insight: Cluster 3 (the tiny but ultra-high-spending cluster containing your primary revenue-driving players) has a massive English-language concentration (~74.5%). Business Action: While localized customer support and marketing are crucial for scale, live-ops events, high-value bundles, VIP customer support, and exclusive community events should be optimized first in English, as this is the primary language used by your most lucrative audience segment.

2. The Netherlands (NL) and Australia (AU) Underrepresent in Casual/Low-Engagement Segments

Insight: Comparing the distribution of countries across clusters, players from the Netherlands (NL) and Australia (AU) are notably more represented in the active, highly engaged, and high-retention clusters (such as Cluster 1 and Cluster 3) than they are in the low-engagement "one-and-done" group (Cluster 0). Business Action: NL and AU represent highly profitable acquisition markets where players exhibit strong structural retention. You should consider shifting more of your paid advertising budget (UAC/Ad Network bids) toward these regions, as an install here has a higher baseline probability of converting into an active, long-term player.

3. The Philippines (PH) is a Volume Play, but Suffers from Lower Baseline Engagement

Insight: Philippines (PH) makes up a very large portion of the overall dataset, but it is heavily concentrated in the lower-retention, lower-engagement clusters (Cluster 0 and Cluster 2). Business Action: Players in Southeast Asian markets like the Philippines are highly responsive to organic and viral acquisition channels (bringing in massive installation volume), but they exhibit lower monetization and progression metrics. To capture value here, introduce monetization models that don't rely purely on high direct-spend, such as rewarded ad placements or lower-priced regional microtransactions tailored to purchasing power.

4. Nordic Languages (Swedish, Danish, Finnish) are Resilient Retention Drivers

Insight: When looking at the LANGUAGE_CAT distribution, Swedish, Danish, and Finnish devices remain remarkably stable or increase their share as we transition from the inactive Cluster 0 to the highly active Cluster 1. This suggests that despite smaller population sizes, players using Nordic-language devices show excellent engagement with the gameplay loop. Business Action: Double down on localization quality checks for these languages. Ensuring perfect in-game translations, cultural nuance adaptation, and localized push notifications for Northern European players will protect this highly stable, highly engaged core community.

6.5. DBSCAN: Parameter Selection and Model Fit

Code cell 66 · saved execution 38

```
from sklearn.neighbors import NearestNeighbors
#Calculate optimal eps
k = 14
neighbors = NearestNeighbors(n_neighbors=k)
```

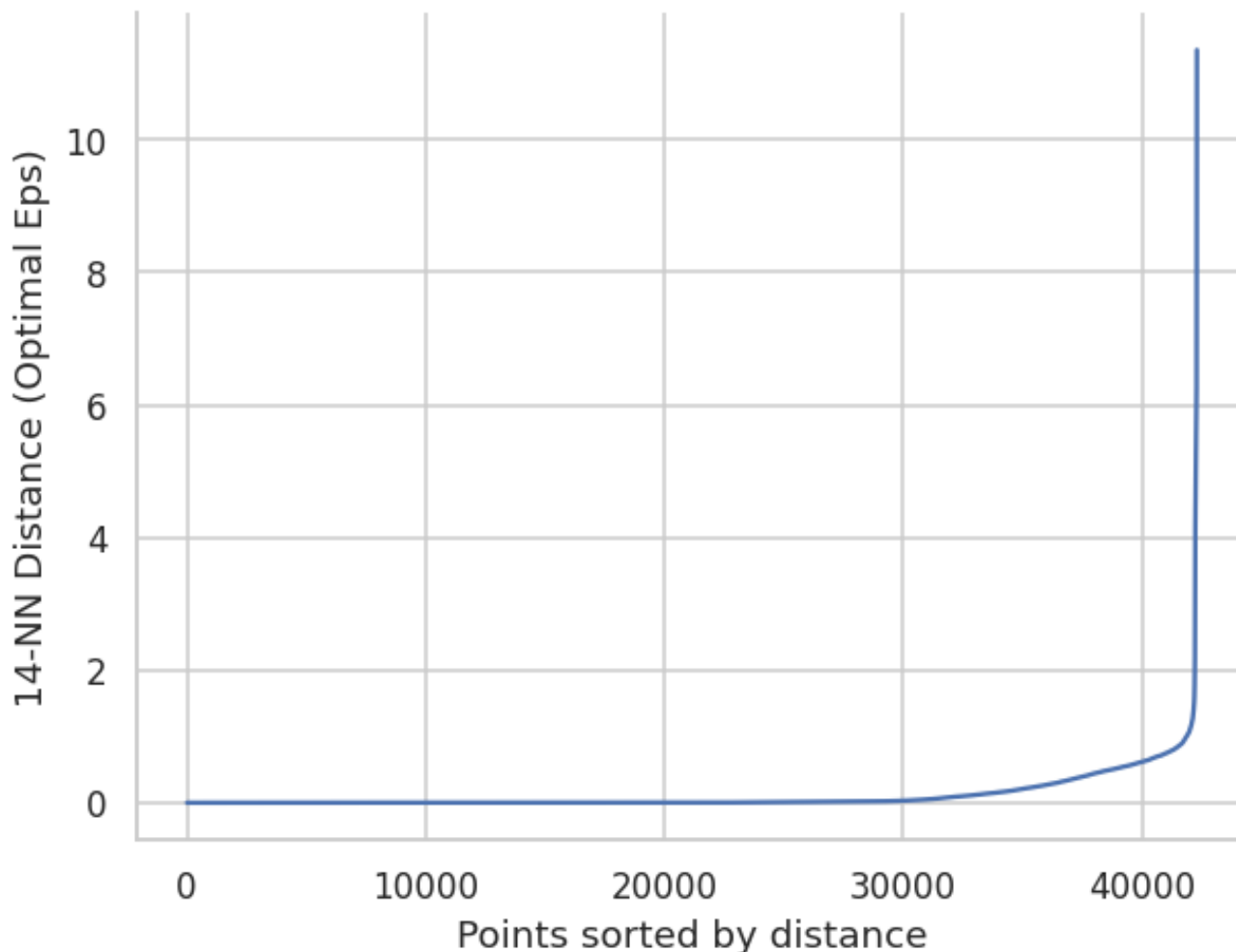
```

neighbors_fit = neighbors.fit(X_scaled)
distances, indices = neighbors_fit.kneighbors(X_scaled)

# Sort distance values by the 14th nearest neighbor distance
distances = np.sort(distances[:, k-1], axis=0)

plt.plot(distances)
plt.xlabel("Points sorted by distance")
plt.ylabel(f"{k}-NN Distance (Optimal Eps)")
plt.show()

```



Code cell 67 · saved execution 39

```

from sklearn.cluster import DBSCAN
from sklearn.metrics import silhouette_score
import numpy as np

# Initialize DBSCAN with chosen parameters
dbs = DBSCAN(eps=1, min_samples=14)

# Fit DBSCAN to the scaled data
dbs.fit(X_scaled)

# Assign cluster labels to the cleaned_df
# Noise points are labeled as -1 by DBSCAN
cleaned_df['dbscan_cluster'] = dbs.labels_

# Print the cluster distribution
print("DBSCAN Cluster Distribution:")
print(pd.Series(dbs.labels_).value_counts().sort_index())

# Calculate number of unique clusters, excluding noise (-1 label)
n_clusters_dbscan = len(np.unique(dbs.labels_)) - (1 if -1 in dbs.labels_ else 0)
print(f"\nNumber of unique DBSCAN clusters (excluding noise): {n_clusters_dbscan}")

# Calculate silhouette score for DBSCAN if there is more than one cluster (excluding noise)
if n_clusters_dbscan > 1:
    # Filter out noise points for silhouette score calculation
    non_noise_indices = dbs.labels_ != -1
    if np.sum(non_noise_indices) > 1: # Ensure at least 2 non-noise points for silhouette_score
        dbs_silhouette_score = silhouette_score(X_scaled[non_noise_indices], dbs.labels_[non_noise_indices])
        print(f"DBSCAN Silhouette Score (excluding noise): {dbs_silhouette_score:.3f}")
    else:

```

```

        print("Not enough non-noise points to calculate DBSCAN Silhouette Score.")
    else:
        print("DBSCAN found 0 or 1 cluster (excluding noise), cannot calculate Silhouette Score.")

DBSCAN Cluster Distribution:
-1      191
 0     24561
 1     12148
 2       5382
 3         14
Name: count, dtype: int64

Number of unique DBSCAN clusters (excluding noise): 4
DBSCAN Silhouette Score (excluding noise): 0.477

```

DBSCAN Parameter Guidance

The choice of `eps` and `min_samples` is crucial for DBSCAN. If you get too many noise points (-1 label) or too few clusters, you might want to adjust these parameters:

*** To reduce noise and get larger, fewer clusters:** Increase `eps` or `min_samples`. *** To get more, smaller clusters:** Decrease `eps` or `min_samples`.

A common way to estimate a good `eps` value is to use a k-distance graph, which involves calculating the distance to the k-th nearest neighbor for each point and plotting these distances in ascending order to find an 'elbow' point. If you would like to explore this, let me know!

6.6. DBSCAN: Cluster Profiles

Code cell 70 · saved execution 40

```

cleaned_df['D1_RETAINED'] = (cleaned_df['D1_TOTAL_SESSION_CT'] > 0).astype(int)
dbscan_plot_cols = feat_cols + ['D1_RETAINED']

dbscan_clusters = sorted(cleaned_df['dbscan_cluster'].unique())
dbsc_colors = [plt.cm.tab10(i) for i in range(len(dbscan_clusters))] # use new colors or adapt existing
dbsc_sizes = cleaned_df['dbscan_cluster'].value_counts().sort_index()

# ----- 1. Mean per cluster (± SEM), one panel per metric -----
# Changed grid size from (2, 3) to (3, 3) to support 8 metrics
fig, ax = plt.subplots(3, 3, figsize=(18, 12))
ax = ax.ravel()

for i, col in enumerate(dbscan_plot_cols):
    g = cleaned_df.groupby('dbscan_cluster')[col]
    means, sems = g.mean(), g.sem()

    bars = ax[i].bar([str(k) for k in dbscan_clusters], means.values, yerr=sems.values,
                     capsize=4, color=dbsc_colors)
    ax[i].bar_label(bars, fmt='%.2f' if col == 'D1_RETAINED' else '%.1f',
                   padding=3, fontsize=9)
    ax[i].set_title(col + (' (retention rate)' if col == 'D1_RETAINED' else ''))
    ax[i].set_xlabel('Cluster')
    ax[i].set_ylabel('Mean')
    ax[i].set_xticks(range(len(dbscan_clusters)))
    ax[i].set_xticklabels([f'{k}\n(n={dbsc_sizes.get(k, 0):,})' for k in dbscan_clusters])
    ax[i].set_ylim(0, (means + sems).max() * 1.2)

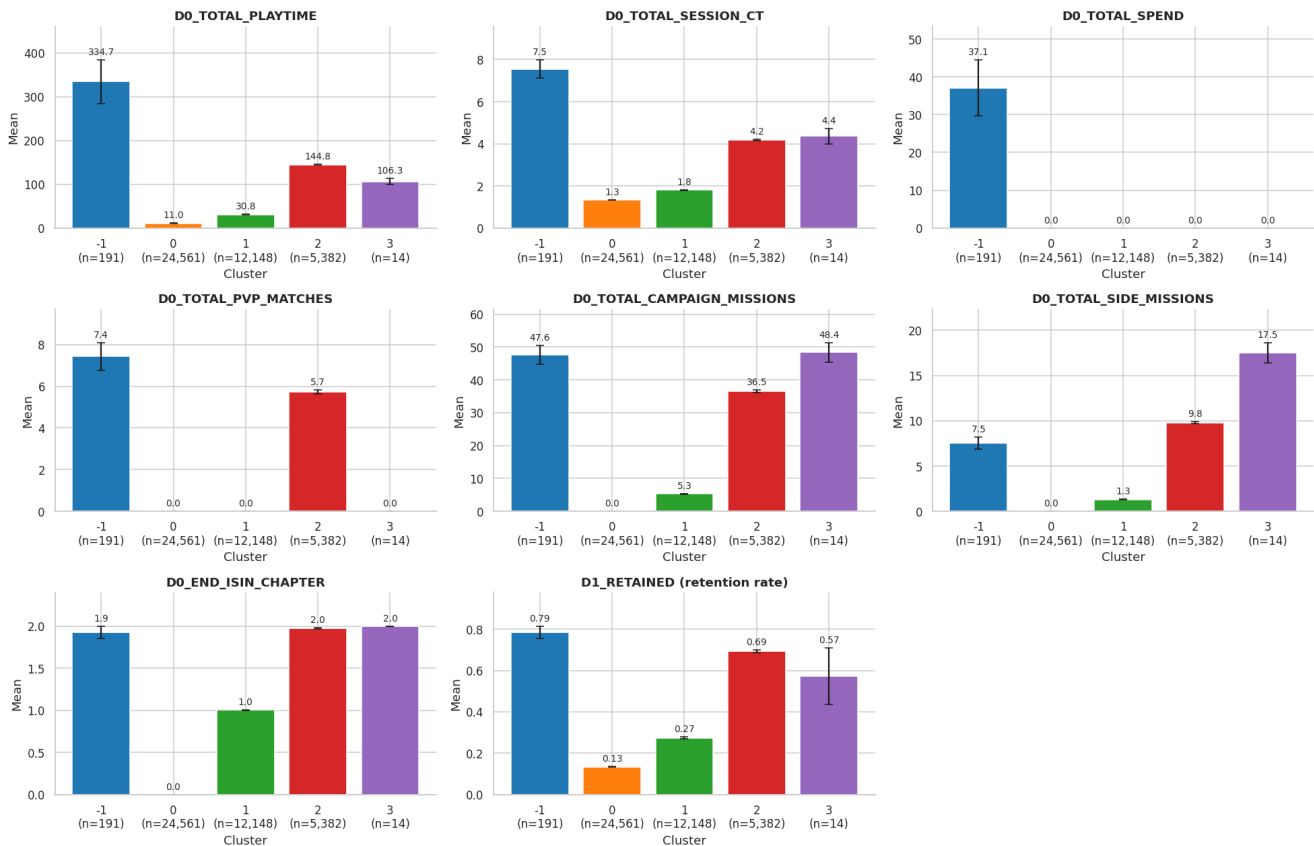
# Hide any unused subplots (the 9th subplot at index 8)
for j in range(len(dbscan_plot_cols), len(ax)):
    ax[j].axis('off')

fig.suptitle('DBSCAN Cluster comparison across key metrics (mean ± SEM)', y=1.02, fontsize=13)
plt.tight_layout()
plt.show()

# ----- 2. Medians, since the means are skewed -----
display(cleaned_df.groupby('dbscan_cluster')[dbscan_plot_cols].median())

```

DBSCAN Cluster comparison across key metrics (mean ± SEM)



	D0_TOTAL_PLAYTIME	D0_TOTAL_SESSION_CT	D0_TOTAL_SPEND	\
dbscan_cluster				
-1	231.750	6.0	0.0	
0	3.730	1.0	0.0	
1	21.225	1.0	0.0	
2	110.375	3.0	0.0	
3	101.335	4.0	0.0	

	D0_TOTAL_PVP_MATCHES	D0_TOTAL_CAMPAIGN_MISSIONS	\
dbscan_cluster			
-1	4.0	37.0	
0	0.0	0.0	
1	0.0	4.0	
2	2.0	30.0	
3	0.0	45.5	

	D0_TOTAL_SIDE_MISSIONS	D0_END_ISIN_CHAPTER	D1_RETAINED
dbscan_cluster			
-1	3.0	2.0	1.0
0	0.0	0.0	0.0
1	0.0	1.0	0.0
2	8.0	2.0	1.0
3	16.0	2.0	1.0

DBSCAN Profiles: Interpretation

Cluster -1 (Noise): These are the points that DBSCAN identified as noise. They tend to have higher playtime, session count, campaign missions, and PVP matches compared to some other clusters, and a 100% retention rate in the median table, indicating they are likely active users that don't fit neatly into dense clusters. This is important to note as DBSCAN often identifies valuable outliers as noise. Cluster 0: This cluster has very low values across all activity metrics (playtime, session count, campaign missions, PVP matches) and a 0% D1 retention rate, suggesting these are largely inactive or one-time users. Cluster 1: This cluster shows slightly more activity than Cluster 0 in terms of playtime and campaign missions, with a 0% retention rate. They seem to engage a bit more than cluster 0 but do not retain. Cluster 2: This cluster exhibits moderate activity, particularly in playtime and campaign missions, and a 100% D1 retention rate. This indicates a segment of retained, moderately engaged users. Cluster 3: This small cluster also shows a 100% D1 retention rate and relatively high engagement in playtime and campaign missions, similar to Cluster 2. They could represent a highly engaged, retained user segment.

7. Overall Findings and Conclusions

Overall, the analysis shows that **early player engagement is strongly associated with D1 retention and progression** in this all Android player dataset. Retained players generally have more sessions, longer playtime, and greater Campaign activity than players who do not return on D1. Early spending is uncommon, but **83.6% of early spenders were retained on D1**, compared with about **24.6% of non-early spenders**, showing a strong association between early spending and retention while not implying that spending causes retention.

The segmentation and clustering analyses also identify distinct player behavior patterns. The K-Means clusters range from a very low-engagement group with **11.9% D1 retention** to highly active groups with about **73–76% D1 retention**. The small high-spending cluster also shows very high Campaign and PVP activity. These results suggest that early behavioral measures such as sessions, playtime, Campaign activity, PVP activity, and spending can meaningfully distinguish different types of players and provide insight.

The Campaign progression results support the same pattern: players who return on D1 generally progress a lot farther through the game. For example, by D30, **80.4% of retained spenders had reached at least Chapter 3**, compared with **32.0% of retained non-spenders** and only **1.2% of lost non-spenders**. Overall, the analysis indicates that **early engagement and progression are important characteristics associated with player retention** in this dataset. Since the dataset contains **Android players only**, these findings should be interpreted within the Android player population represented in the data only, and should not be applied to other platforms.

Appendix A. Original Colab Export Utilities

These original cells install export tools and download files in Colab. They are not analysis steps.

Code cell 74 · saved execution 47

```
# 1. Install system tools required for PDF generation
!apt-get update
!apt-get install -y texlive-xetex texlive-fonts-recommended texlive-plain-generic weasyprint > /dev/null

# 2. Search for the notebook inside Google Drive and local directories

0% [Working]

Hit:1 https://cli.github.com/packages stable InRelease

0% [Connected to cloud.r-project.org (99.84.132.23)] [Connecting to r2u.stat.il

Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease

0% [Waiting for headers] [Connecting to r2u.stat.illinois.edu (192.17.190.167)]

Hit:3 https://cloud.r-project.org/bin/linux/ubuntu noble-cran40/ InRelease

0% [Connected to r2u.stat.illinois.edu (192.17.190.167)] [Connecting to ppa.lau

Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:6 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:7 https://r2u.stat.illinois.edu/ubuntu noble InRelease
Hit:8 https://ppa.launchpadcontent.net/deadsnakes/ppa/ubuntu noble InRelease
Reading package lists... Done
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu
noble InRelease' does not seem to provide it (sources.list entry misspelt?)
Converting /content/drive/MyDrive/Gunadhi_Vivienne_OCMC_Final.ipynb to PDF...
[NbConvertApp] Converting notebook /content/drive/MyDrive/Gunadhi_Vivienne_OCMC_Final.ipynb to pdf
[NbConvertApp] Writing 45866 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 57305 bytes to /content/drive/MyDrive/Gunadhi_Vivienne_OCMC_Final.pdf
Trying alternative HTML-to-PDF compilation...
[NbConvertApp] Converting notebook /content/drive/MyDrive/Gunadhi_Vivienne_OCMC_Final.ipynb to html
[NbConvertApp] Writing 347527 bytes to /content/drive/MyDrive/Gunadhi_Vivienne_OCMC_Final.html
```

Code cell 75 · saved execution 48

```
import os
from google.colab import files

search_paths = ['.', '/content/drive/MyDrive', '/content/drive/MyDrive/Colab Notebooks']
found_notebooks = ['Copy of EDA_fixing_version_with_interpretations.ipynb']

for path in search_paths:
    if os.path.exists(path):
        for f in os.listdir(path):
            if f.endswith('.ipynb') and not f.startswith('.'):
                found_notebooks.append(os.path.join(path, f))

if not found_notebooks:
```

```

    print("To export: Go to 'File' -> 'Download' -> 'Download .ipynb', then drag and drop it here to convert it
instantly to PDF!")
else:
    # Select the first notebook match
    target_notebook = found_notebooks[0]
    print(f"Converting {target_notebook} to PDF...")
    !jupyter nbconvert --to pdf "{target_notebook}"
    pdf_name = os.path.basename(target_notebook).replace('.ipynb', '.pdf')
    if os.path.exists(pdf_name):
        files.download(pdf_name)
    else:
        # Alternate HTML conversion to avoid LaTeX issues
        print("Trying alternative HTML-to-PDF compilation...")
        html_name = os.path.basename(target_notebook).replace('.ipynb', '.html')
        !jupyter nbconvert --to html "{target_notebook}"
        if os.path.exists(html_name):
            !weasyprint "{html_name}" "{pdf_name}"
            if os.path.exists(pdf_name):
                files.download(pdf_name)

Converting Copy of EDA_fixing_version_with_interpretations.ipynb to PDF...
[NbConvertApp] WARNING | pattern 'Copy of EDA_fixing_version_with_interpretations.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

```

Options

=====

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases.

To see all configurable class-options for some <cmd>, use:

```
<cmd> --help-all
```

```

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]

--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]

--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]

--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]

-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]

--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the error message in the
cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was
specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin
    read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace
    Run nbconvert in place, overwriting the existing notebook (only
        relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook
--FilesWriter.build_directory=]

--clear-output
    Clear output of current file and save in place,
        overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook
--FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]

--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook
--FilesWriter.build_directory= --CoalesceStreamsPreprocessor.enabled=True]

--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_prompt=True]

--no-input
    Exclude input cells and output prompts from converted document.
    This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exclude_input=True
--TemplateExporter.exclude_input_prompt=True]

--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]

--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]

--show-input

```

Shows code input. This flag is only useful for dejavu users.
Equivalent to: [--TemplateExporter.exclude_input=False]

--embed-images
Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides exports.
Equivalent to: [--HTMLExporter.embed_images=True]

--sanitize-html
Whether the HTML in Markdown cells and cell outputs should be sanitized..
Equivalent to: [--HTMLExporter.sanitize_html=True]

--log-level=<Enum>
Set the log level by value or name.
Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
Default: 30
Equivalent to: [--Application.log_level]

--config=<Unicode>
Full path of a config file.
Default: ''
Equivalent to: [--JupyterApp.config_file]

--to=<Unicode>
The export format to be used, either one of the built-in formats
['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf']
or a dotted object name that represents the import path for an
``Exporter`` class
Default: ''
Equivalent to: [--NbConvertApp.export_format]

--template=<Unicode>
Name of the template to use
Default: ''
Equivalent to: [--TemplateExporter.template_name]

--template-file=<Unicode>
Name of the template file to use
Default: None
Equivalent to: [--TemplateExporter.template_file]

--theme=<Unicode>
Template specific theme(e.g. the name of a JupyterLab CSS theme distributed as prebuilt extension for the lab template)
Default: 'light'
Equivalent to: [--HTMLExporter.theme]

--sanitize_html=<Bool>
Whether the HTML in Markdown cells and cell outputs should be sanitized. This should be set to True by nbviewer or similar tools.
Default: False
Equivalent to: [--HTMLExporter.sanitize_html]

--writer=<DottedObjectName>
Writer class used to write the results of the conversion
Default: 'FilesWriter'
Equivalent to: [--NbConvertApp.writer_class]

--post=<DottedOrNone>
PostProcessor class used to write the results of the conversion
Default: ''
Equivalent to: [--NbConvertApp.postprocessor_class]

--output=<Unicode>
Overwrite base name use for output files.
Supports pattern replacements '{notebook_name}'.
Default: '{notebook_name}'
Equivalent to: [--NbConvertApp.output_base]

--output-dir=<Unicode>
Directory to write output(s) to. Defaults to output to the directory of each notebook. To recover previous default behaviour (outputting to the current working directory) use . as the flag value.
Default: ''
Equivalent to: [--FilesWriter.build_directory]

--reveal-prefix=<Unicode>
The URL prefix for reveal.js (version 3.x).
This defaults to the reveal CDN, but can be any url pointing to a copy of reveal.js.
For speaker notes to work, this must be a relative path to a local copy of reveal.js: e.g., "reveal.js".
If a relative path is given, it must be a subdirectory of the current directory (from which the server is run).
See the usage documentation (<https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slideshow>) for more details.
Default: ''
Equivalent to: [--SlidesExporter.reveal_url_prefix]

--nbformat=<Enum>
The nbformat version to write.
Use this to downgrade notebooks.
Choices: any of [1, 2, 3, 4]
Default: 4
Equivalent to: [--NotebookExporter.nbformat_version]

Examples

The simplest way to use nbconvert is

```

> jupyter nbconvert mynotebook.ipynb --to html

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python',
'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

> jupyter nbconvert --to latex mynotebook.ipynb

Both HTML and LaTeX support multiple output templates. LaTeX includes
'base', 'article' and 'report'. HTML includes 'basic', 'lab' and
'classic'. You can specify the flavor of the format used.

> jupyter nbconvert --to html --template lab mynotebook.ipynb

You can also pipe the output to stdout, rather than a file

> jupyter nbconvert mynotebook.ipynb --stdout

PDF is generated via latex

> jupyter nbconvert mynotebook.ipynb --to pdf

You can get (and serve) a Reveal.js-powered slideshow

> jupyter nbconvert myslides.ipynb --to slides --post serve

Multiple notebooks can be given at the command line in a couple of
different ways:

> jupyter nbconvert notebook*.ipynb
> jupyter nbconvert notebook1.ipynb notebook2.ipynb

or you can specify the notebooks list in a config file, containing::

    c.NbConvertApp.notebooks = ["my_notebook.ipynb"]

> jupyter nbconvert --config mycfg.py

To see all available configurables, use `--help-all`.

```

Trying alternative HTML-to-PDF compilation...

```

[NbConvertApp] WARNING | pattern 'Copy of EDA_fixing_version_with_interpretations.ipynb' matched no files
This application is used to convert notebook files (*.ipynb)
to various other formats.

```

WARNING: THE COMMANDLINE INTERFACE MAY CHANGE IN FUTURE RELEASES.

Options

=====

The options below are convenience aliases to configurable class-options, as listed in the "Equivalent to" description-line of the aliases. To see all configurable class-options for some <cmd>, use:

```

<cmd> --help-all

--debug
    set log level to logging.DEBUG (maximize logging output)
    Equivalent to: [--Application.log_level=10]

--show-config
    Show the application's configuration (human-readable format)
    Equivalent to: [--Application.show_config=True]

--show-config-json
    Show the application's configuration (json format)
    Equivalent to: [--Application.show_config_json=True]

--generate-config
    generate default config file
    Equivalent to: [--JupyterApp.generate_config=True]

-y
    Answer yes to any questions instead of prompting.
    Equivalent to: [--JupyterApp.answer_yes=True]

--execute
    Execute the notebook prior to export.
    Equivalent to: [--ExecutePreprocessor.enabled=True]

--allow-errors
    Continue notebook execution even if one of the cells throws an error and include the error message in the
    cell output (the default behaviour is to abort conversion). This flag is only relevant if '--execute' was
    specified, too.
    Equivalent to: [--ExecutePreprocessor.allow_errors=True]

--stdin
    read a single notebook file from stdin. Write the resulting notebook with default basename 'notebook.*'
    Equivalent to: [--NbConvertApp.from_stdin=True]

--stdout
    Write notebook output to stdout instead of files.
    Equivalent to: [--NbConvertApp.writer_class=StdoutWriter]

--inplace
    Run nbconvert in place, overwriting the existing notebook (only
    relevant when converting to notebook format)
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook
--FilesWriter.build_directory=]

--clear-output
    Clear output of current file and save in place,
    overwriting the existing notebook.
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook
--FilesWriter.build_directory= --ClearOutputPreprocessor.enabled=True]

```

```

--coalesce-streams
    Coalesce consecutive stdout and stderr outputs into one stream (within each cell).
    Equivalent to: [--NbConvertApp.use_output_suffix=False --NbConvertApp.export_format=notebook]
--FilesWriter.build_directory= --CoalesceStreamsPreprocessor.enabled=True]
--no-prompt
    Exclude input and output prompts from converted document.
    Equivalent to: [--TemplateExporter.exclude_input_prompt=True --TemplateExporter.exclude_output_prompt=True]
--no-input
    Exclude input cells and output prompts from converted document.
    This mode is ideal for generating code-free reports.
    Equivalent to: [--TemplateExporter.exclude_output_prompt=True --TemplateExporter.exclude_input=True]
--TemplateExporter.exclude_input_prompt=True]
--allow-chromium-download
    Whether to allow downloading chromium if no suitable version is found on the system.
    Equivalent to: [--WebPDFExporter.allow_chromium_download=True]

--disable-chromium-sandbox
    Disable chromium security sandbox when converting to PDF..
    Equivalent to: [--WebPDFExporter.disable_sandbox=True]
--show-input
    Shows code input. This flag is only useful for dejavu users.
    Equivalent to: [--TemplateExporter.exclude_input=False]
--embed-images
    Embed the images as base64 dataurls in the output. This flag is only useful for the HTML/WebPDF/Slides
    exports.
    Equivalent to: [--HTMLExporter.embed_images=True]
--sanitize-html
    Whether the HTML in Markdown cells and cell outputs should be sanitized..
    Equivalent to: [--HTMLExporter.sanitize_html=True]
--log-level=<Enum>
    Set the log level by value or name.
    Choices: any of [0, 10, 20, 30, 40, 50, 'DEBUG', 'INFO', 'WARN', 'ERROR', 'CRITICAL']
    Default: 30
    Equivalent to: [--Application.log_level]
--config=<Unicode>
    Full path of a config file.
    Default: ''
    Equivalent to: [--JupyterApp.config_file]
--to=<Unicode>
    The export format to be used, either one of the built-in formats
    ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng',
    'rst', 'script', 'slides', 'webpdf']
    or a dotted object name that represents the import path for an
    ``Exporter`` class
    Default: ''
    Equivalent to: [--NbConvertApp.export_format]
--template=<Unicode>
    Name of the template to use
    Default: ''
    Equivalent to: [--TemplateExporter.template_name]
--template-file=<Unicode>
    Name of the template file to use
    Default: None
    Equivalent to: [--TemplateExporter.template_file]
--theme=<Unicode>
    Template specific theme(e.g. the name of a JupyterLab CSS theme distributed
    as prebuilt extension for the lab template)
    Default: 'light'
    Equivalent to: [--HTMLExporter.theme]
--sanitize_html=<Bool>
    Whether the HTML in Markdown cells and cell outputs should be sanitized.This
    should be set to True by nbviewer or similar tools.
    Default: False
    Equivalent to: [--HTMLExporter.sanitize_html]
--writer=<DottedObjectName>
    Writer class used to write the
    results of the conversion
    Default: 'FilesWriter'
    Equivalent to: [--NbConvertApp.writer_class]
--post=<DottedOrNone>
    PostProcessor class used to write the
    results of the conversion
    Default: ''
    Equivalent to: [--NbConvertApp.postprocessor_class]
--output=<Unicode>
    Overwrite base name use for output files.
    Supports pattern replacements '{notebook_name}'.
    Default: '{notebook_name}'
    Equivalent to: [--NbConvertApp.output_base]
--output-dir=<Unicode>
    Directory to write output(s) to. Defaults
    to output to the directory of each notebook. To recover
    previous default behaviour (outputting to the current
    working directory) use . as the flag value.
    Default: ''
    Equivalent to: [--FilesWriter.build_directory]
--reveal-prefix=<Unicode>
    The URL prefix for reveal.js (version 3.x).
    This defaults to the reveal CDN, but can be any url pointing to a copy
    of reveal.js.
    For speaker notes to work, this must be a relative path to a local
    copy of reveal.js: e.g., "reveal.js".

```

If a relative path is given, it must be a subdirectory of the current directory (from which the server is run).
See the usage documentation
(<https://nbconvert.readthedocs.io/en/latest/usage.html#reveal-js-html-slideshow>)
for more details.

Default: ''

Equivalent to: [--SlidesExporter.reveal_url_prefix]

--nbformat=<Enum>

The nbformat version to write.

Use this to downgrade notebooks.

Choices: any of [1, 2, 3, 4]

Default: 4

Equivalent to: [--NotebookExporter.nbformat_version]

Examples

The simplest way to use nbconvert is

```
> jupyter nbconvert mynotebook.ipynb --to html
```

Options include ['asciidoc', 'custom', 'html', 'latex', 'markdown', 'notebook', 'pdf', 'python', 'qtpdf', 'qtpng', 'rst', 'script', 'slides', 'webpdf'].

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

```
> jupyter nbconvert --to latex mynotebook.ipynb
```

Both HTML and LaTeX support multiple output templates. LaTeX includes 'base', 'article' and 'report'. HTML includes 'basic', 'lab' and 'classic'. You can specify the flavor of the format used.

```
> jupyter nbconvert --to html --template lab mynotebook.ipynb
```

You can also pipe the output to stdout, rather than a file

```
> jupyter nbconvert mynotebook.ipynb --stdout
```

PDF is generated via latex

```
> jupyter nbconvert mynotebook.ipynb --to pdf
```

You can get (and serve) a Reveal.js-powered slideshow

```
> jupyter nbconvert myslides.ipynb --to slides --post serve
```

Multiple notebooks can be given at the command line in a couple of different ways:

```
> jupyter nbconvert notebook*.ipynb
> jupyter nbconvert notebook1.ipynb notebook2.ipynb
```

or you can specify the notebooks list in a config file, containing::

```
c.NbConvertApp.notebooks = ["my_notebook.ipynb"]
```

```
> jupyter nbconvert --config mycfg.py
```

To see all available configurables, use '--help-all'.

Code cell 76 · saved execution 50

```
import os
from google.colab import files
import shutil

# Target only the requested notebook
target_notebook = '/content/drive/MyDrive/Colab Notebooks/Copy of EDA_fixing_version_with_interpretations.ipynb'

if not os.path.exists(target_notebook):
    print(f"■ Could not find the notebook at: {target_notebook}")
else:
    print(f"■ Selected notebook for conversion: {target_notebook}")

# 2. Convert selected notebook to PDF
print("\nConverting to PDF... Please wait...")
!jupyter nbconvert --to pdf "{target_notebook}"

notebook_dir = os.path.dirname(target_notebook)
base_name = os.path.basename(target_notebook).replace('.ipynb', '.pdf')
possible_pdf_paths = [
    os.path.join(notebook_dir, base_name),
    os.path.join('.', base_name)
]

found_pdf = None
for path in possible_pdf_paths:
    if os.path.exists(path):
        found_pdf = path
        break

if found_pdf:
    local_pdf = os.path.basename(found_pdf)
    if found_pdf != os.path.join('.', local_pdf):
        shutil.copy(found_pdf, local_pdf)
```

```

print(f"\n■ Success! Downloading your PDF: {local_pdf}")
files.download(local_pdf)
else:
    # Fallback to HTML + Weasyprint conversion if LaTeX has compilation issues
    print("\n■ PDF compilation failed or took too long. Trying alternative HTML-to-PDF conversion...")
    html_name = os.path.basename(target_notebook).replace('.ipynb', '.html')
    !jupyter nbconvert --to html "{target_notebook}"

    possible_html_paths = [
        os.path.join(notebook_dir, html_name),
        os.path.join('.', html_name)
    ]

    found_html = None
    for path in possible_html_paths:
        if os.path.exists(path):
            found_html = path
            break

    if found_html:
        local_pdf_fallback = os.path.basename(target_notebook).replace('.ipynb', '.pdf')
        !weasyprint "{found_html}" "{local_pdf_fallback}"
        if os.path.exists(local_pdf_fallback):
            print(f"\n■ Success via HTML fallback! Downloading: {local_pdf_fallback}")
            files.download(local_pdf_fallback)
        else:
            print("■ HTML fallback conversion to PDF failed.")
    else:
        print("■ Conversion failed. HTML target could not be found.")

■ Selected notebook for conversion: /content/drive/MyDrive/Colab Notebooks/Copy of
EDA_fixing_version_with_interpretations.ipynb

Converting to PDF... Please wait...
[NbConvertApp] Converting notebook /content/drive/MyDrive/Colab Notebooks/Copy of
EDA_fixing_version_with_interpretations.ipynb to pdf
[NbConvertApp] Support files will be in Copy of EDA_fixing_version_with_interpretations_files/
[NbConvertApp] Making directory ./Copy of EDA_fixing_version_with_interpretations_files
[NbConvertApp] Writing 287275 bytes to notebook.tex
[NbConvertApp] Building PDF
[NbConvertApp] Running xelatex 3 times: ['xelatex', 'notebook.tex', '-quiet']
[NbConvertApp] Running bibtex 1 time: ['bibtex', 'notebook']
[NbConvertApp] WARNING | bibtex had problems, most likely because there were no citations
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 1388452 bytes to /content/drive/MyDrive/Colab Notebooks/Copy of
EDA_fixing_version_with_interpretations.pdf

■ Success! Downloading your PDF: Copy of EDA_fixing_version_with_interpretations.pdf

<IPython.core.display.Javascript object>

<IPython.core.display.Javascript object>

```